Highlights

Deep Green AI: Energy Efficiency of Deep Learning across Language–Framework Ecosystems

Leonardo Pampaloni,Marco Pagliocca,Enrico Vicario,Roberto Verdecchia

- One specification, 92 automated checks: 7 stacks run the same experiment
- 210 runs, 12600 blocks; training energy spans 7.4×–9.8× across stacks
- One exported module, 3 stacks: spread 1.1×–1.6× against 9.8× across all 7
- The estimator's duration field, not the stacks, produced "faster is not greener"
- 75 % of blocks are timed over a longer window than their energy

Deep Green AI: Energy Efficiency of Deep Learning across Language–Framework Ecosystems

Leonardo Pampaloni[a,*], Marco Pagliocca[a,*], Enrico Vicario[a] and Roberto Verdecchia[a]

[a] *University of Florence, Florence, Italy*

ARTICLE INFO

Keywords:
Machine Learning
Deep Learning
Green AI
Energy Efficiency
Sustainability
Green IT

ABSTRACT

Context. Deep Learning's environmental footprint raises a practical question: does the choice of language–framework ecosystem carry an energy price? Answering it requires that every ecosystem run the same experiment and be measured the same way — neither of which is verifiable from reported energy figures alone.

Objectives. We quantify how ecosystem choice affects the energy efficiency of DL training and inference, and, inseparably, what an apparatus must guarantee before such a comparison means anything.

Methods. Two convolutional architectures (ResNet-18, VGG-16) across 7 ecosystems — Python with PyTorch, TensorFlow and JAX; C++ with LibTorch; Java with Deeplearning4j; R with `torch`; Rust with `tch` — on three datasets. A written specification defines what "the same experiment" means across stacks and 92 automated checks enforce it against the source. Energy is read from hardware counters (NVML and RAPL) with CodeCarbon over the same phase boundaries as a second reading. Each configuration runs 5 independent times, interleaved for 206 of the 210 runs, and every stack records per-epoch accuracy.

Results. Over 210 runs and 12600 doubly instrumented blocks, training energy differs across ecosystems by 7.4× to 9.8× depending on the block, with a median between-run coefficient of variation of 0.49 %. Among the 3 stacks that load the same exported TorchScript module on the same pinned backend build the spread is 1.1×–1.6×, so most of the effect is between stacks that do not share the module rather than within those that do. Inference is wider still, 23.1× to 38.7×, on blocks short enough that the apparatus is part of what is being measured. On Fashion-MNIST the stacks converge to within 0.5 percentage points; on the harder datasets they do not, and energy must be read alongside accuracy. Replication is what makes such a comparison auditable at all: the first execution of this design lost 12 of 105 VGG-16 runs to chance accuracy at full energy cost, which we traced to the initialisers the frameworks ship under one architecture name. With every stack started from one canonical seeded export and matched initialisers, 0 of the 105 VGG-16 runs in this campaign reached chance. On the apparatus, the estimator's energy agrees with the counters to 0.3 % because on this platform it reads the same registers — but 75 % of blocks are reported with a duration seconds longer than the interval their energy was accumulated over, so power derived from those two fields is understated by up to 13.0× on the shortest blocks and exact on the longest.

Conclusions. Ecosystem selection materially affects the energy profile of DL applications, but the size and even the direction of that effect depend on decisions in the measurement apparatus that are invisible in the resulting data. We report the comparison together with the specification, the conformance checker and the raw dual-instrument records needed to audit it.

1. Introduction

The rapid advancement of Artificial Intelligence (AI), particularly in the domain of Deep Learning (DL), has delivered groundbreaking progress across domains such as healthcare, natural language processing, environmental monitoring, and autonomous systems. As models grow in size and complexity, their performance continues to improve, often reaching or surpassing human-level capabilities in specific tasks. However, these achievements have come at a growing environmental cost. The training and deployment of state-of-the-art neural networks demand vast computational resources and energy, contributing to significant carbon footprints [1, 2, 3, 4, 5, 6, 7]. Recent studies show that energy usage and emissions vary widely depending on model architecture, dataset size, hardware infrastructure, and even geographic location of execution [8, 9]. In response, standardized benchmarks, such as MLPerf Tiny, have begun to incorporate energy measurements to assess efficiency under realistic deployment conditions [10]. In a world of limited environmental resources, the ever-growing energy required to power AI has become a pressing concern, making sustainability a central dimension of AI research [11, 12].

Rather than framing this trend solely as a cause of concern, the field is now embracing a proactive paradigm, referred to as *Green AI* [12, 13]. This research line promotes the development of AI systems that are not only accurate and effective but also energy-conscious and efficient. Recent surveys have highlighted advances across lightweight architectures, efficient training and inference strategies, model compression, and hardware-aware optimizations [4, 14, 15, 16]. Evidence increasingly points in the direction of energy improvements not necessarily implying a degradation of accuracy [17].

*Corresponding authors: Leonardo Pampaloni and Marco Pagliocca.
`leonardo.pampaloni1@edu.unifi.it` (L. Pampaloni); `marco.pagliocca@edu.unifi.it` (M. Pagliocca); `enrico.vicario@unifi.it` (E. Vicario); `roberto.verdecchia@unifi.it` (R. Verdecchia)
ORCID(s): `0000-0001-9206-6637` (R. Verdecchia)

Beyond algorithmic efficiency, research in software sustainability has shown that the choice of programming language itself can have a substantial impact on energy usage [18, 19, 20, 21, 22, 23, 24], and on the run time and memory that partly determine it [25]. Nevertheless, while both Green AI and the environmental sustainability of programming languages have been independently investigated, their intersection remains largely unexplored. A first step was made by Georgiou *et al.* [26], who analyzed the energy footprint of Deep Learning frameworks within Python, and by Marini *et al.* [27], who studied the impact of programming languages in AI workloads and state explicitly that they "purposely opted not to focus on GPU-intensive algorithms such as deep neural networks, leaving their consideration as future work". That is the gap this study addresses.

For a practitioner, however, a programming language is inseparable from its *ecosystem*. Choosing a language for DL typically implies adopting a specific software stack, including language bindings, frameworks/libraries, runtimes/execution engines, and low-level driver/toolchain dependencies. As a result, in real-world settings it is often more meaningful to evaluate *language–framework ecosystems* as they are adopted in practice, rather than attempting to isolate a "pure" language effect under synthetic, fully standardized conditions.

Prior work has approached this from three directions without closing it. Energy has been compared across DL frameworks *within* one language ecosystem, including at the GPU and runtime-infrastructure level [26, 28]; language-level energy has been measured for CPU-bound general-purpose workloads [18, 21]; and the *runtime* cost of cross-language DL bindings has been characterised for time, memory and accuracy, but not for energy [29]. What has not been measured is energy across multiple language-based DL stacks for GPU training and inference, where frameworks, execution engines, toolchains and dataset complexity all vary together [30].

This paper addresses this gap by presenting an empirical study on *the impact of language–framework ecosystems on the energy efficiency of DL workloads*. To achieve our goal, we consider two widely adopted convolutional neural networks (ResNet-18 and VGG-16) across 7 language–framework ecosystems, spanning five host languages (Python, C++, Java, R and Rust) and the frameworks each of them offers in practice (PyTorch, TensorFlow and JAX; LibTorch; Deeplearning4j; and the `torch` and `tch` bindings), on three heterogeneous datasets (Fashion-MNIST, CIFAR-100 and Tiny ImageNet).

A cross-ecosystem energy comparison, however, rests on two premises that are rarely stated and never visible in the resulting numbers: that every stack is running the same experiment, and that every stack is measured the same way. We found both to be far harder to satisfy than the literature suggests, and we came to that conclusion the expensive way. An earlier campaign of ours, of exactly this shape, was executed carefully and produced an entirely coherent story; auditing it against its own source code afterwards showed that the stacks had drifted apart in learning rate, input shape, evaluation batching and data-loader parallelism, that the measurement tool had been configured five different ways across them, and that two stacks were capable of running the whole workload on the CPU while reporting nothing but a log line. None of this was visible in the energy figures. All of it moved them.

We therefore treat the apparatus as part of the contribution rather than as a preliminary. Three commitments follow. *(i)* A written experiment specification states what "the same experiment" means across stacks — model, optimisation, data pipeline, backend, measurement and replication — and 92 automated checks enforce it against the source code, so that a divergence fails a check instead of quietly changing a number. *(ii)* Energy is read from hardware counters, NVML's accumulated-energy register for the accelerator and the RAPL package-energy counters for the CPU, with CodeCarbon running over the same phase boundaries as a second reading; every block therefore carries two readings that can be held against each other. *(iii)* Each configuration is executed as 5 independent, interleaved runs with distinct seeds, so that the run rather than the epoch is the unit of analysis, and every ecosystem records per-epoch test accuracy, so that energy can be normalised by the quality actually reached rather than assumed equal.

Our results are correspondingly more modest and more defensible than a bare ranking. Between-run repeatability is high — the median coefficient of variation of training energy across 210 runs is 0.49 % — so the intervals we report, Student-*t* on five independent runs, describe between-run uncertainty rather than epoch autocorrelation. Under a common measurement boundary, training energy differs across ecosystems by 7.4× to 9.8× depending on architecture and dataset, with C++ cheapest in every ResNet-18 block and the cheapest VGG-16 stack changing with the dataset, while final test accuracy converges to within 0.5 percentage points on Fashion-MNIST across all 7 stacks. On the two harder datasets the stacks do not converge to a common accuracy, and there energy must be read alongside quality rather than instead of it. The ecosystem effect is real and large; on Fashion-MNIST it is energetic rather than qualitative.

Replication earned its cost in a way we did not anticipate. In the first execution of this design, 12 of 105 VGG-16 runs converged to exactly chance accuracy and stayed there for the full epoch budget, consuming 10.7 % of that campaign's training energy and producing nothing. We had read that as a property of the ecosystems; it was the weight initialiser, which frameworks ship differently under one architecture name (Section 6.2.1). Starting every stack from one canonical seeded export and matching the initialisers of the stacks that build their own layers, 0 of 105 runs collapsed here, with an exact one-sided 95 % upper bound of 4.2 % on the susceptible cells. Replication is what made the defect visible and what retired it; a design with one run per configuration reports whichever outcome its single seed produced.

The two readings agree on *energy* to 0.3 % over 12600 blocks — on this platform the estimator reads the same two registers we do, so its energy is not an estimate at all, and that relocates the difficulty with software-based estimation rather than dissolving it. What does not agree is time. The duration CodeCarbon reports beside each energy figure is not the interval the energy was accumulated over: 75 % of blocks carry seconds of tracker lifetime that no energy was drawn in, in 3 discrete modes that block length predicts but does not fully determine. Power derived by dividing the one field by the other is therefore understated by up to 13.0× on the shortest blocks and exact on the longest — a bias that lands squarely on the fastest ecosystems and on the inference phase, which is to say on precisely the comparisons such a study exists to make.

The contributions of this work are as follows:

- A systematic empirical comparison of DL energy consumption across 7 language–framework ecosystems for GPU-based workloads, replicated 5 times per configuration and reported with between-run uncertainty and quality normalisation;
- An experiment specification for cross-ecosystem energy studies, together with 92 executable conformance checks that verify it against the source, the built binaries and the campaign's own metric files, and a single measurement contract implemented by all 7 stacks;
- A dual-instrument characterisation of CodeCarbon against hardware energy counters over 12600 measurement blocks, including a quantified failure mode of its reported duration that invalidates energy–time analyses built on it;
- A comprehensive replication package, including all data, scripts, and settings required to fully reproduce our results.[1]

Improving the energy efficiency of AI systems is not just a technical challenge but also an ethical imperative as the field matures and scales [31]. In this regard, providing concrete empirical evidence on the energy consumption of different language–framework ecosystems represents a crucial first step towards the development of AI that is not only powerful, but also truly sustainable.

2. Background

The notion of *Green AI* [12, 13, 17] promotes the development of AI systems that are not only accurate but also energy-efficient and sustainable. Several indicators can be used to quantify sustainability, such as carbon footprint, financial cost, or energy usage [7, 5]. In this work we focus on **energy consumption**, measured in Joules, as it represents a direct and reproducible measure of computational effort. Unlike carbon footprint, which is heavily influenced by the energy mix and geographic location of execution, energy consumption can be compared more consistently across heterogeneous DL stacks under a uniform measurement protocol.

We treat execution time as a secondary metric rather than a proxy for energy. The prevailing position in the software-energy literature is that lower runtime does not imply lower energy [30, 32, 18], and we test it directly in Section 6.4 rather than assuming it. That position is itself under revision: van Kempen *et al.* [33] re-analyse language–energy rankings of this kind and argue that much of the reported effect is an artefact of how the comparison was constructed. Our argument is adjacent but not the same. Theirs concerns the confounds in what is being compared; ours concerns a field of the instrument's output, and we arrive at it from the opposite direction — our counter-bracketed measurements make time a *good* proxy for energy in this workload, where the estimator's own duration field makes it a bad one. Our result differs from that literature, and the difference is instructive: on counter-bracketed durations, energy and time rank our configurations almost identically. Those studies largely concern CPU-bound general-purpose benchmarks, where a language implementation can trade instruction count against memory traffic and shift energy per second substantially. In a GPU-bound DL workload the accelerator dominates and holds a narrow band of power, so time and energy are close to proportional. We report this as a scope condition on the established finding, not a refutation of it — and we show in Section 6.4 that a study of exactly our shape can obtain either answer depending on which field of the instrument's output it reads as the duration.

Deep Learning workloads are typically divided into two distinct phases: training and inference. *Training* requires iterative updates of model parameters through backpropagation and is generally the most computationally expensive stage. *Inference*, by contrast, involves only forward passes on a trained model and is usually less resource demanding. Both phases, however, display different energy profiles—*i.e.* optimizations in one phase do not necessarily carry over to the other [14, 15]. This motivates the need to analyze the two phases separately when evaluating their energy efficiency.

Language–framework ecosystems may influence the energy efficiency of DL workloads through a combination of factors, including framework execution engines, runtime overheads, data input pipelines, numerical precision policies, and the underlying driver/toolchain configuration. While compiled and interpreted languages differ in their execution models, in GPU-based DL workloads much of the computation is typically delegated to optimized backend kernels; therefore, differences observed in practice should be interpreted at the *ecosystem level* rather than attributed to the programming language alone. Moreover, some ecosystems introduce additional runtime layers, which can affect both performance and energy behavior; this study does not attribute its measured spread to that mechanism (Section 6.2). Understanding these ecosystem-level differences is crucial, since language choice is often treated as a secondary engineering decision, yet in practice it constrains the set of

[1] Replication package: `https://github.com/Pampaj7/DeepGreen`. The Zenodo record `https://zenodo.org/records/17734884` archives the earlier campaign and is superseded by this one.

available frameworks, runtimes, and supported toolchains, ultimately affecting the energy profile of DL model training and inference.

Energy measurement for workloads of this kind can be read at three places, and the choice of place matters more than the choice of tool. A *wall meter* sees the whole machine including power supply losses, fans and drives. *Hardware energy counters* — NVML's `nvmlDeviceGetTotalEnergyConsumption` on NVIDIA accelerators and RAPL on the CPU package — accumulate energy in the silicon itself and are read by software but not estimated by it. RAPL is Intel's interface by origin, but AMD exposes an equivalent set of model-specific registers and Linux presents both through the same `powercap` tree, whose nodes are named `intel-rapl` whatever the vendor. The machine used here is AMD, so the name in the sysfs path is not the name of the silicon. *Software estimators* such as `CodeCarbon` [34] sit on top of whatever the platform exposes and substitute a model wherever it exposes nothing.

Software estimators have become standard in sustainability-oriented AI research [31, 8], and are relied upon as the sole instrument across model compression analysis [35], optimizer efficiency benchmarks [36], large-scale evaluations of Large Language Models [37] and cybersecurity systems [38] — studies whose conclusions inherit whatever the estimator gets wrong, without a second reading that could reveal it. Comparative studies have also run `CodeCarbon` alongside hardware wattmeters in the same setup and report that software estimates track sensor readings closely under controlled conditions while deviating more on heterogeneous infrastructure [39, 40]. Independent validation against external ground truth across hundreds of AI experiments nonetheless reports estimator errors of up to 40%, and systematic underestimation of 20–30% in some settings [41].

That literature asks whether an estimator's number is close to the truth. We take a different and complementary position. The question is not whether a software estimator is accurate enough to be trusted, but whether a study can tell when it is not — and the two lead to different remedies. If the problem is accuracy, a better estimator or a correction factor fixes it. If the problem is that an estimator silently substitutes a model for a measurement, or pads a duration it reports as measured, no correction factor helps. An estimator that falls back on a model when a counter is unavailable, and announces the substitution only in a log line, produces a number of the same shape and plausibility either way. We therefore instrument every measurement block *twice*: hardware counters as the primary instrument, and `CodeCarbon` started and stopped at the same phase boundaries as a second reading. Agreement between the two is then an observable property of each block rather than an assumption, and the places where they cannot agree — CodeCarbon's RAM term is derived from installed memory capacity, and no counter on this platform can confirm or refute it — are identified rather than absorbed.

This also fixes the boundary. NVML plus RAPL is a *board-and-package* measurement: the accelerator card including its memory, plus the CPU package, excluding the power supply, fans, drives and host DRAM that a wall meter would see. It must not be presented as a whole-system figure. We had no wall meter available and say so, reporting a board-and-package quantity consistently rather than a mixture of measured and modelled terms that corresponds to no physical boundary at all. Section 6.5 reports what the two instruments say about each other across the campaign.

3. Related Work

Table 1 places this study against the work closest to it on the two axes that matter here: how many language ecosystems are compared, and how energy is measured.

Two of these deserve more than a row. Alizadeh and Castor [28] compare runtime infrastructures for the same models on the same GPU, and reach the ecosystem-level question from inside Python: their treatment is the runtime beneath one language, ours is the language–framework stack above it. Li *et al.* [29] compare TensorFlow and PyTorch bindings across five languages, which is our independent variable exactly, and measure run time, memory and accuracy rather than energy — the gap this study fills, and the reason we treat the measurement boundary as a contribution rather than a preliminary.

Jay *et al.* [43] compare software power meters, CodeCarbon among them, against physical meters on CPU and GPU, and establish that a measurement window must comfortably exceed a *sampling* meter's interval. We report that our own design does not exhibit that constraint, and why: on this platform the estimator reads accumulating registers rather than sampling, and its energy residual against those registers is flat across every block length (Section 6.5). What we add is orthogonal — a characterisation of the *duration* an estimator reports beside its energy, which to our knowledge has not been examined.

4. Research Methodology

This work adopts an empirical methodology grounded in the principles of software engineering empirical experimentation [45, 46, 47, 48]. Our objective is not to propose a new algorithm or framework, but to systematically analyze how choosing among language–framework ecosystems (including their supported runtime and toolchain) can impact the energy efficiency of DL workloads. To this end, we designed an *in vitro* controlled experiment that treats ecosystems as independent variables, DL workloads as experimental objects, and energy consumption as the primary dependent variable [49, 50, 51].

Research goal and questions. Following the Goal-Question-Metric (GQM) paradigm [49], our goal is formulated as:

Analyze DL training and inference,
for the purpose of quantifying and comparing

Table 1
The closest prior work, on the two axes this study varies. "Stacks" counts distinct language–framework combinations compared under one protocol.

Study	Stacks	Hardware	Energy instrument	Replication
Li *et al.* [42]	4 frameworks, Python only	CPU + GPU	external meter	repeated
Pereira *et al.* [18]	27 languages, no DL	CPU	RAPL	10 runs
Gordillo *et al.* [21]	8 languages, no DL	CPU	hardware meter	repeated
Georgiou *et al.* [26]	2 frameworks, Python only	CPU + GPU	RAPL + NVML	30 runs
Alizadeh & Castor [28]	4 runtimes, Python only	CPU + GPU	software estimator	repeated
Li *et al.* [29]	5 bindings, 2 frameworks	CPU + GPU	none (time, memory, accuracy)	repeated
Jay *et al.* [43]	software meters, not stacks	CPU + GPU	meters against a wattmeter	repeated
Marini *et al.* [27]	5 languages	CPU + GPU	software estimator	single run
Chattaraj *et al.* [44]	2 languages, no DL	CPU	software estimator	repeated
This study	7 stacks, 5 languages	CPU + GPU	NVML + RAPL counters, estimator alongside	5 runs

their energy consumption,
with respect to the choice of DL ecosystems (i.e. language bindings, frameworks/libraries, supported runtimes/execution engines, and associated toolchains),
from the viewpoint of software engineering researchers and practitioners,
in the context of computer vision image classification workloads.

Research questions. This study investigates how language–framework ecosystem selection influences the energy efficiency of Deep Learning (DL) workloads. We ask four questions:

- **RQ1:** *How does language–framework ecosystem choice affect the energy efficiency of DL* training*?*
- **RQ2:** *How does language–framework ecosystem choice affect the energy efficiency of DL* inference*?*
- **RQ3:** *How do energy efficiency and execution time relate across different DL ecosystems?*
- **RQ4:** *To what extent are the answers to RQ1–RQ3 determined by the measurement apparatus rather than by the ecosystems?*

RQ4 is not a robustness check appended to the other three. A cross-ecosystem energy comparison is an inference from an instrument to a claim about software, and the inference is only as good as the guarantee that the instrument reads the same quantity, over the same window, for every stack. We therefore treat the apparatus as an object of measurement in its own right: every block is recorded twice over the same boundaries, and RQ4 asks what those readings say about each other and about the answers to RQ1–RQ3.

Objects of study. We focus on convolutional neural networks (CNNs), which remain fundamental in modern computer vision tasks [4, 14, 15]. Specifically, we consider two canonical architectures: ResNet-18 [52] and VGG-16 [53]. These models are selected not only for their distinct computational characteristics (residual connections vs. sequential depth), but primarily because they are natively provided *off-the-shelf* by the vast majority of DL frameworks. By relying on these standard, pre-implemented architectures, we ensure that our measurements capture the frameworks' internal optimizations and native performance, rather than introducing researcher-specific biases that would result from implementing the models from scratch.

To account for varying workload complexity, we employ three widely utilized datasets. *Fashion-MNIST* provides a lightweight baseline with 28×28 grayscale images across 10 classes [54], and is often used as a modern drop-in replacement for MNIST[2]. It includes 60,000 training and 10,000 test images. *CIFAR-100* increases complexity with 32×32 color images across 100 categories, remaining a canonical benchmark for image classification [55], and comprises 50,000 training and 10,000 test images. *Tiny ImageNet* represents a demanding workload with 64×64 color images across 200 classes and 100,000 training and 10,000 test images, serving as a proxy for ImageNet-scale tasks [56].

All datasets are resized to a uniform resolution of 32×32 pixels once, offline, before any stack sees them: `scripts/normalise_dataset_resolution.py` rewrites every image in the three datasets at that resolution, so each stack's own resize is a no-op and no framework's resampling filter can enter the comparison. Before the change, 4 of the 7 stacks decoded measurably different pixels on Tiny ImageNet — the one dataset that has to be downsampled — with means that agreed and variances that did not, which is the signature of a resampling filter rather than of different content. This preserves the relative differences in complexity (grayscale vs. color, number of classes, dataset size) while removing input resolution as a confounding factor [27, 26]. In addition, standardizing input size avoids the need to modify the final classification layer of the networks for each dataset, thus keeping model architectures fixed across experiments. This design choice not only improves comparability of results but also reflects realistic developer practice, where pre-trained or canonical models are often reused with

[2] `https://github.com/zalandoresearch/fashion-mnist`

minimal architectural modifications [17, 13]. Finally, a common resolution improves stability across frameworks, some of which (e.g. `tch` for Rust, Deeplearning4j for Java) offer more reliable performance with standardized image sizes [18, 21]. Nevertheless, fixing a 32×32 input resolution may interact with framework-specific memory and kernel selection strategies; therefore, conclusions should not be extrapolated to higher-resolution workloads without additional experiments.

Independent and dependent variables. Our primary independent variable is the *DL ecosystem*, i.e. the language–framework stack as adopted in practice (including its officially supported runtime and CUDA/cuDNN toolchain). In our design, each ecosystem is treated as a single categorical treatment; we do not attempt to decompose effects into separate contributions of language, framework, runtime, or toolchain. Specifically, we evaluate 7 language-based ecosystems: Python (`PyTorch`, `TensorFlow`, `JAX`), C++ (`LibTorch`), Java (`Deeplearning4j`), R (`torch`), and Rust (`tch`). A second independent variable is the *DL phase* (training and inference).

MATLAB with the Deep Learning Toolbox was part of an earlier campaign and is out of scope here. The reason is specific rather than dismissive: the toolbox does not expose the training loop at the granularity the specification of Section 4.1 requires, so its data pipeline, its evaluation batching and its measurement boundaries cannot be brought into conformance with the other 7. Including a stack that provably runs a different experiment would reintroduce exactly the confound this design exists to remove.

Our dependent variables are the *energy consumption* of each measurement block, in Joules at a board-and-package boundary (accelerator card plus CPU package), and the *test accuracy* reached at the end of each epoch. Recording the second alongside the first is what allows energy to be normalised by achieved quality in Section 6.2; a fixed-budget comparison that does not record accuracy cannot distinguish an efficient stack from one that has simply learned less [5, 31, 6].

Study design. We adopt a *black-box design*: models are executed using off-the-shelf implementations under identical hyperparameters (e.g. learning rate, batch size) [20, 19, 32]. This choice enforces the realism of the study by reflecting how practitioners use frameworks *in vivo*, rather than relying on ad-hoc or reimplemented versions.

By keeping hyperparameters constant, we implement a controlled *fixed-budget* comparison across ecosystems (i.e. identical training recipe and number of epochs). This design ensures a fair baseline comparison in which we minimize the risk of introducing researcher-specific implementation biases, thereby improving comparability; however, it does not guarantee convergence equivalence across frameworks. This represents an explicit trade-off: allowing per-ecosystem hyperparameter tuning could yield better absolute efficiency for each stack, but would confound whether differences stem from the ecosystem itself or from the tuning process.

Moreover, we treat framework-level execution engines and optimizations (e.g. graph compilation, runtime schedulers, backend kernels) as *part of the ecosystem* rather than normalizing them away. As a consequence, observed differences should be interpreted as ecosystem-level effects, not as isolated language-level behavior.

Numerical precision policy. All 7 ecosystems run under one precision policy, and it is set rather than inherited: fp32 storage and arithmetic, with TF32 permitted on the accelerator's tensor cores for convolution and for matrix multiplication. For convolution that is what cuDNN does on this hardware unless it is told otherwise, so it is the path a practitioner measures without choosing it; for matrix multiplication it is not, since every binding leaves that path at fp32 by default, and we state both halves rather than set one and inherit the other. A single variable carries the policy — `DEEPGREEN_TF32`, applied through the frameworks for the stacks that import our harness and through `NVIDIA_TF32_OVERRIDE` for the four processes that never do — and every run's manifest records the flags that resulted. Setting it to `0` denies TF32 to all 7. Mixed precision is a separate axis and is enabled in no stack here.

The policy belongs to the specification of Section 4.1 rather than to each ecosystem, and it is there because of what happened when it was not. A first execution of this campaign pinned TF32 *off* in the shared harness; the pin reached one stack of 7 and no other, and that single difference was worth more than any difference between bindings this study measures. Section 6.2.2 reports it as an ablation.

4.1. What "the same experiment" means, and how it is enforced

A fixed-budget comparison presumes that the budget is the same. In practice each ecosystem is written by a different person against a different API, and the places where they can silently diverge are numerous, individually reasonable and invisible in the energy output: a default learning rate that differs between two framework APIs, a data loader whose worker count follows the machine rather than the protocol, an evaluation loop written one image at a time because the API made that natural, an input tensor left at its native resolution in one stack and resized in another.

We therefore wrote the protocol down before running anything, as a specification with six parts, and made conformance machine-checkable.

- **S1 — Model.** One canonical definition per architecture. `scripts/export_torchscript_models.py` exports one TorchScript module per (architecture, dataset) pair, and `models/MANIFEST.json` records that module's parameter count and a hash of the parameters in name order. The 3 ecosystems that share the pinned LibTorch build (Python/PyTorch, C++, Rust) load that module; the remaining stacks build the same architecture in their own framework, from the same initialiser and the same batch-normalisation convention, because a framework's defaults for either are not a property of the ecosystem a reader is asking about. Every stack asserts its own parameter count at startup against the manifest, carried into the run as `DEEPGREEN_EXPECTED_PARAMS`, and refuses to train if it does not match. Counting parameters is the weak form of the check — two networks can agree on a total and differ layer by layer — so `scripts/verify_architecture_parity.py` builds each stack's model for every block and compares the sorted multiset of parameter tensor shapes, which is comparable across languages where names and orderings are not: all 7 stacks agree, shape for shape, on all six blocks. R links a LibTorch of its own and its binding cannot switch a script module between training and evaluation mode, so it builds the architecture rather than loading the module; it belongs to the LibTorch lineage but not to the control group. The previous version of this clause asserted the parameter check and did not implement it; Section 8 records what that cost.

- **S2 — Optimisation.** One optimiser, one learning rate, one batch size, one epoch count, one loss and one seeding policy. The learning rate and the batch size are checked against the source of every file the campaign executes — universally, rather than by finding one file that matches; the rest is asserted by the specification and was checked by reading.
- **S3 — Pipeline.** One input specification: $32 \times 32 \times 3$ images in raw $[0, 1]$ with no per-stack normalisation, resized once offline so that no framework's resampler enters the comparison; a bounded and equal number of loader workers; batched evaluation at the training batch size; and the whole 10,000-image test split evaluated in every stack, with no final partial batch dropped and no step count rounded down. Every run writes a fingerprint of the split it decoded — mean, standard deviation and range over its 30720000 pixel values — so the pipelines are demonstrated to agree rather than asserted to.
- **S4 — Backend.** One LibTorch build across the 3 stacks that share it, and one precision policy for all 7: fp32 storage, TF32 allowed, stated to the CUDA libraries campaign-wide. The accelerator is asserted against the built artefact and not only against the source: `readelf -d` on every Rust binary we build must show a CUDA library among its `NEEDED` entries, and the harness refuses to start on a CPU device rather than warning about it.
- **S5 — Measurement.** One measurement bridge, one sampling interval, one tracking mode, one set of counters, one unit, and one per-epoch record — training loss, test loss and test accuracy — persisted by every stack alongside energy, checked against the campaign's own metric files rather than against the source that writes them. Training accuracy is recorded by the 2 stacks whose training loops already compute it, and is read by no analysis in this paper; the schema names those stacks rather than implying 7.
- **S6 — Replication.** Each configuration is executed 5 times as independent processes with distinct seeds, interleaved rather than run back to back. The seed is threaded from the run contract into each stack's data order and its initialisation, and two of the three ways a stack can ignore the seed it is handed are now checked against the stacks' source rather than against the campaign planner alone.

The specification is enforced by 92 automated checks that read the source of every ecosystem, the configuration of every tracker, the built binaries and the campaign's own metric files, and fail the build when a stack drifts out of conformance. This matters more than it may appear. A specification that lives only in a paper is a description of intent; the same specification expressed as an executable check is a property of the artefact, and it fails at the moment of the divergence rather than at peer review. 12 of the defects catalogued in Section 8 were introduced by us while building or repairing this study; most were caught by a mechanism rather than by inspection, and two were not, which is the subject of what follows. Several are now enforced by checks under S1, S3 and S5, so the same divergence cannot recur.

What the checks caught, and what they could not. Of the 92 checks, 92 pass and 0 fail. That is not the interesting number, and we would not report it on its own. Building this round's suite meant first discovering that several of its checks could not fail. The learning-rate check was existential where the specification is universal: it passed when one file in a glob matched, so one stack's check passed while four sibling files carried other learning rates, citing as its evidence a file the campaign does not execute — a reviewer who set `lrAdam = 1e-2` and `batchSize = 999` in the classes that do run still got `[ok]`. A guard on the output directory could fail only in the case where the check before it had already failed. And the device check verified that a definition string appeared in a source file, which a binary with no CUDA library linked into it satisfies perfectly. The first is universal now and the second resolves both paths against a temporary directory, and each fails on the defect that motivated it; the device clause keeps its source check and has gained an artefact check beside it. We report this rather than quietly regenerating a count, because a specification whose checks are relaxed until they pass is a description of intent again — and a check that cannot fail is the same thing with a green light on it.

The limit is sharper than that, and the campaign is what found it. The suite proved that the architectures matched, that the pixels matched and that the protocol matched, and none of that requires the code to run. Fifteen hours into the campaign 4 runs had failed and all of them were the same ecosystem and architecture: JAX training VGG-16 raised `flax.errors.InvalidRngError` on its first batch, because the shared classifier head carries a dropout layer and that stack's training step never handed it a random stream. The parameter count it had already asserted was correct, since dropout has no parameters, and every check this revision added passed. A conformance suite built from source text cannot establish that a stack takes one step; only running it can. We fixed the defect and replayed those runs (Section 4.2), and we keep the lesson rather than the embarrassment: static enforcement is a floor, not a ceiling, and the campaign is part of the apparatus that checks the campaign.

Residual, documented divergences. Three could not be removed and are reported rather than hidden. First, Deeplearning4j 1.0.0-M2.1 is the last release of its API and links against the CUDA 11.6 runtime, so the Java stack cannot be brought onto the CUDA 12 toolchain the other six share. Second, and more consequentially, that stack resolves its convolutions without cuDNN and cannot do otherwise at this version: they go through nd4j's own im2col and cuBLAS. `readelf -d` on `libnd4jcuda.so` shows a `NEEDED` entry for cuBLAS and none for cuDNN, and the cuDNN convolution helper has no release at any version, so nothing in the stack can call the cuDNN libraries its own redistributable ships. The specification document in the replication package carries the rest of the evidence. It is the only stack of 7 that reaches no cuDNN kernel, and a conformance check now watches that library and fails if a future version gains one, so the claim cannot quietly stop being true. Because it cannot be fixed, we bound it: disabling cuDNN in a stack that has it, holding hardware, model and data constant, costs 13.07× the energy and 16.42× the time on ResNet-18 at batch 128. That is an upper bound on what losing cuDNN can cost rather than an estimate of this stack's own penalty — nd4j's im2col path is not PyTorch's non-cuDNN fallback — but it is large enough that Java's cost in this study is in part a statement about a missing convolution backend rather than about a language, and we repeat the qualification wherever that cost is reported. Third, the R `torch` binding cannot switch a script module between training and evaluation mode, so R alone cannot consume the shared TorchScript module of S1 and builds the same architecture instead; its shapes are verified against the other six inside the campaign environment, which is the only place its binding loads. All three are properties of the ecosystems, and arguably part of what one measures when one measures an ecosystem; we name them because a reader deciding what a cost should be attributed to needs to know which of them are in play.

4.2. Measurement procedure

Energy is read from hardware counters at the boundaries of each measurement block: NVML's `nvmlDeviceGetTotalEnergyConsumption`, an accumulating millijoule register on the accelerator, and the RAPL package-energy counters exposed at `/sys/class/powercap` for the CPU, with wraparound at `max_energy_range_uj` handled explicitly. The reported quantity is their sum, identical for every ecosystem.

The two halves of that sum are not at the same boundary, and the sum is not "the chip". NVML's register is a *board* quantity: it includes the GPU die, its GDDR6X and the card's voltage regulators. RAPL's package domain is the CPU package alone. Host DRAM is in neither — this platform exposes no `dram` RAPL domain, so its energy, of order ten watts, is not measurable here and is absent from every figure we report. We name the quantity board-and-package rather than chip-level, and we note the asymmetry with our own criticism of the estimator's modelled RAM term: that term is unverifiable, and ours is missing altogether.

`CodeCarbon` [34] runs over the same window as a second reading, in machine tracking mode at a one-second sampling interval and the default `api_call_interval` of 8, configured identically for all 7 stacks through a single shared bridge. We name that last parameter because Section 6.4 shows it is the one that decides whether a reported duration is a duration. The four non-Python ecosystems drive that bridge over a line protocol with synchronous acknowledgement, so that a phase boundary in the measured process and the corresponding counter read cannot drift apart; Section 5 says what an asynchronous version of it cost.

A *measurement block* is one phase of one epoch: 12600 of them in total, each carrying two energy readings, a duration, and the test accuracy reached. We reserve the word for that unit. The six (model, dataset) combinations we report results over are *cells*, and the twelve (model, dataset, phase) groups the statistical tests run over are *test groups*. Because tracking is machine-wide, no other work runs on the host during a campaign; blocks measured while a background download was in progress during development were discarded and the protocol amended rather than corrected after the fact.

Replication and the unit of analysis. Each of the 42 configurations is executed 5 times as an independent process with a distinct seed, giving 210 runs. 206 of them ran *interleaved* rather than consecutively, so that drift in machine state — thermal, background, driver clock behaviour — is spread across conditions instead of aliasing onto whichever ecosystem happened to run last. The remaining 4, all JAX on VGG-16, are the runs the dropout defect above stopped at their first batch; they were replayed 2 days later on the same host, and they are the one place where interleaving does not hold. Section 9 reports what a between-window calibration says that is worth. All uncertainty we report is between-run: the 30 epochs within a run share an initialisation, a JIT outcome, an allocator state and a thermal trajectory, and treating them as repeated measurements would understate uncertainty while inflating apparent significance.

Quality normalisation. Because every stack records test accuracy per epoch, we report energy in three forms: energy per epoch, energy per unit of accuracy achieved, and the energy required to first reach a target accuracy fixed per dataset. The last is the closest to a practitioner's question, and is the one a fixed-budget design cannot answer on its own.

Reproducibility and rigor. Each run receives a seed from the run contract, and that seed reaches both the data order and the model initialisation in every stack. Two of the three ways a stack can ignore the seed it is handed are now checked against the stacks' own source, which is two more than the previous version of this study checked: in an earlier campaign of this design 3 stacks of 7 were handed a distinct seed per repetition and did not use it — one hardcoded its shuffle seed, so all 5 repetitions saw the same data order; one never forwarded the seed to its loader; and one rebuilt its generator inside the epoch loop and replayed a single permutation for every epoch — while the conformance check reported five distinct seeds, having asked the campaign planner rather than the stacks. Initialisation is no longer each framework's default either: under S1 every stack draws convolutions and dense layers from the same distributions, because a default initialiser is not a property of the ecosystem this study is asking about. Nevertheless, full determinism cannot be guaranteed across all stacks due to non-deterministic GPU kernels and differences in backend execution engines; this is one reason the design replicates rather than relying on a single execution per configuration.

A complete replication package is provided (see replication package link in Section 1), including: (i) source code for all implementations, (ii) scripts to reproduce runs, (iii) environment specifications (e.g. virtual environments, Maven, CMake), and (iv) collected raw data [17, 13]. Configurations are standardized across languages where feasible, with deviations documented when framework-specific requirements arise [18, 21]. This package enables replication, validation, and extension under different hardware or software conditions [45, 47].

5. Experimental Execution

This section describes the practical execution of the study, complementing the research design presented in Section 4. Here we document the experimental setup, implementation details, measurement procedure, and reproducibility package, following a step-by-step process to ensure transparency [45, 48].

Experimental setup. All experiments reported here were executed on a single dedicated workstation: an NVIDIA GeForce RTX 3090 (Ampere, 24 GB GDDR6X, 350 W board power limit, driver 595.84), an AMD Ryzen 9 5900X (12 cores / 24 threads), 30 GB of RAM and NVMe storage, running Ubuntu 26.04 LTS (kernel 7.0.0). The machine was otherwise idle for the duration of the campaign, which whole-machine energy tracking requires.

Two properties of this platform bear directly on the measurements. First, both energy counters are available: the RTX 3090 exposes NVML's accumulated-energy register, and the Ryzen's package-energy counters are readable through the Linux `powercap` interface — which, for historical reasons, presents AMD domains under `intel-rapl` names. We read the package domain and handle counter wraparound at `max_energy_range_uj` explicitly. Second, this is a desktop rather than a server: it has one accelerator, one CPU socket and an order of magnitude less RAM than a datacentre node, which is precisely why the modelled RAM term discussed in Section 6.5 is a smaller share of a software estimator's total here than it would be on a large-memory host.

We report this as a replication platform rather than as the platform. An earlier campaign of the same design ran on a dual-socket Intel Xeon Gold 5418Y server with an NVIDIA L40S; absolute energy figures are not comparable between the two machines, and none of the comparisons in this paper is made across them. What carries over is the design, the specification and the instrument characterisation, all of which are properties of the method rather than of the host.

Model implementations. Every ecosystem trains the same architecture, and the stacks whose backend permits it train the same artefact. Under specification S1 the 3 stacks on the pinned LibTorch build — Python/PyTorch, C++ and Rust — do not build 3 independent ResNet-18s and VGG-16s but load one TorchScript module per (architecture, dataset) pair, exported once by a single script and recorded in a manifest with its parameter count and a hash of the parameters in name order. The other four build the same definition in their own framework and assert that count at startup against the same manifest, and a parity script compares all 7 stacks on the sorted multiset of their parameter tensor shapes rather than on a total. That last distinction is not pedantry. In an earlier campaign of this design the stacks were compared on parameter counts that nothing checked, and VGG-16 ran as four different networks spanning an order of magnitude in parameters under one name; comparing them was comparing models, not ecosystems.

Table 2 lists the versions. The Python stacks are installed in three separate virtual environments rather than one, because their CUDA and cuDNN wheels conflict: a shared environment resolved to a cuDNN that TensorFlow could not dlopen, and TensorFlow then ran unaccelerated while reporting only a warning — a failure mode the accelerator assertion of S4 now turns into an error at startup. The assertion by itself was not enough. A binary can satisfy any source-level device check and still have no CUDA library linked into it, which is what happened to our own Rust stack for a day, so S4 is checked against the built artefact as well: `readelf -d` on every released Rust binary, which immediately found two stale ones.

Toolchain divergence, and what is left of it. The 3 stacks of the control group are pinned to one LibTorch build (2.7.0, CUDA 12.8), so within it the backend is genuinely held constant. Four of the 7 ecosystems cannot join them, for three different reasons. TensorFlow and JAX bring their own backends and are not LibTorch stacks at all. Deeplearning4j 1.0.0-M2.1 links against the CUDA 11.6 runtime, resolves its convolutions without cuDNN, and has had no subsequent release; the stack now carries its own redistributables through `org.bytedeco:cuda-platform-redist`, which makes it reproducible without making it contemporary, and Section 4.1 bounds what the missing convolution backend can be worth. R's `torch` 0.17.0 bundles its own LibTorch and resolves CUDA at load time. We report these as documented

Table 2
Ecosystems, frameworks, versions and backends, as executed in the re-executed campaign of Section 5. The 3 stacks marked † share one pinned LibTorch build and load one exported TorchScript module, and form the internal control group of the study. R, marked ‡, links a LibTorch of its own and builds the same architecture: same lineage, different backend, different artefact. Every stack, control group or not, runs under one precision policy — fp32 storage, TF32 allowed, no mixed precision — and asserts its parameter count against one manifest at startup.

Ecosystem	Framework / Library	Version	Backend
Python	PyTorch / TorchVision†	v2.7.0 / v0.22.0	LibTorch 2.7.0, CUDA 12.8
	TensorFlow / `tf_keras`	v2.19.1 / v2.19.0	cuDNN 9.3.0, CUDA 12 wheels
	JAX / Flax / Optax	v0.10.2 / v0.12.8 / v0.2.8	cuDNN 9.24.0, CUDA 12 wheels
C++	LibTorch†	v2.7.0	LibTorch 2.7.0, CUDA 12.8
Java	Deeplearning4j	v1.0.0-M2.1	ND4J CUDA 11.6, cuDNN unused
R	`torch`‡	v0.17.0	bundled LibTorch, CUDA 12
Rust	`tch`†	v0.20.0	LibTorch 2.7.0, CUDA 12.8
Instrumentation			
	CodeCarbon	v3.3.0	identical config, all stacks
	NVML / RAPL counters	driver 595.84	read by the shared harness

divergences rather than normalising them away, and Section 6.2 reports the LibTorch control group separately so that a reader can see how much of the total spread survives when the backend is held fixed.

During a preliminary pilot phase, the Julia programming language (using `Flux` and `Lux`) was also considered. However, persistent technical challenges encountered during implementation related to gradient updates during training, which remained unresolved also after directly contacting the developers of the libraries, prevented its reliable inclusion in the final study.

Procedure. As summarized in Table 3, each experimental configuration (model × dataset × ecosystem) is executed with consistent hyperparameters tuned to balance comparability and realism [4, 14]. Training runs are performed for 30 epochs with a batch size of 128, using the Adam optimizer with a fixed learning rate of 1×10^{-4}. Loss is computed via cross-entropy, consistent across all ecosystems. Input preprocessing includes resizing all images to 32×32 pixels, with grayscale datasets (e.g. Fashion-MNIST) expanded to three channels for compatibility with CNN backbones. These settings reflect canonical practices in the community while ensuring comparability across implementations [10, 18].

Measurement. Each phase of each epoch is bracketed by two counter reads — NVML's accumulated-energy register and the CPU package RAPL counter — and is simultaneously tracked by `CodeCarbon` v3.3.0 [34]. The two are started and stopped by the same code path, so the intervals they *accumulate over* cannot drift apart. The counter reads are nested immediately inside the tracker's, which costs a fixed offset we quantify in Section 6.5 rather than assume away. The interval CodeCarbon *reports* is a different thing again, and Section 6.4 is about that.

The four non-Python ecosystems drive the instrumentation through a single shared bridge: a Python process that accepts `START`, `STOP`, `METRIC` and `EXIT` over a line protocol, reads the counters, and acknowledges each command before the measured process proceeds. The acknowledgement is not incidental. An earlier, asynchronous version of the same bridge allowed a phase boundary in the measured process to run ahead of the corresponding counter read, which under-reported one stack's training energy by 24% and recorded its evaluation energy as exactly zero — a value that looks like a bug only if someone thinks to look at it.

Running the bridge out of process also isolates the measured stack from the instrument: a crash in the tracker cannot take down a six-hour training run. It does not remove the tracker's own cost from the measurement — under whole-machine tracking its once-per-second sampling runs inside the block and its package energy is inside the block's RAPL delta. That cost is a constant per second and identical for every stack, so it does not bias the comparison, but it is inside the numbers and we would rather say so. All runs execute in isolation on an otherwise idle host, as whole-machine tracking requires, and each block is logged with dataset, model, ecosystem, repetition, seed, phase, epoch, both energy readings, the duration and the test accuracy reached.

Reproducibility. A comprehensive replication package accompanies this study (see the link in Section 1). Beyond source code, automation scripts, per-ecosystem environment specifications and raw logs, it contains three artefacts specific to this design:

(i) the **experiment specification** of Section 4.1, written before the campaign and versioned with it;

(ii) the **conformance checker**, 92 executable checks that read the source of every ecosystem and the configuration of every tracker, and fail when a stack drifts out of specification;

(iii) the **dual-instrument records**: every block's counter reading and estimator reading side by side, which is what makes the analysis of Section 6.5 reproducible rather than anecdotal.

Every quantity this manuscript reports from the campaign is generated from those records by the analysis pipeline and injected into the source at build time. The exceptions are

Table 3
Hyperparameter configuration, identical across all ecosystems in the re-executed campaign. The learning rate and the batch size are verified against the source of every file the campaign executes, universally rather than by one file that happens to match; the precision policy is verified as a campaign-wide export and by a negative check that no stack pins it privately; the other rows are asserted by the specification and were checked by reading. Section 4.1 reports why that distinction is not pedantic.

Parameter	Value
Number of epochs	30
Batch size	128
Optimizer	Adam
Learning rate	1×10^{-4}
Loss function	Cross-entropy
Input resolution	32×32, resized once offline; no stack resizes
Input channels	3 (grayscale datasets expanded)
Numerical precision	fp32 storage, TF32 allowed, no mixed precision; one campaign-wide setting
Accelerator	asserted at startup, CPU fall-back refused; the Rust binaries checked for a linked CUDA library
Data loader workers	2, bounded and equal in every stack
Input normalisation	none; raw $[0, 1]$ inputs in every stack and every model
Evaluation	batched at the training batch size, over the whole 10,000-image test split
Seeding	one seed per run from the run contract, reaching data order and initialisation
Independent runs	5 per configuration, 206 of 210 interleaved

stated where they occur: figures recovered from the audit of an earlier campaign, and one-off diagnostics reproduced from the record that established them. Two limits on that provenance are worth stating rather than leaving for a replicator to find. Every run's manifest records the machine state it ran under — driver, power limit, clocks, persistence mode, governor, boost and the precision policy in force — but every run of this campaign predates the harness-provenance field and so carries machine state without a source revision; the field is recorded from the next campaign on. And the analysis used to write its tables into the directory the manuscript reads, so a monitoring pass over a campaign still in flight briefly replaced the campaign's own run count with a partial one; the pipeline now diverts to a separate directory until the expected number of runs is complete. Neither changes a number reported here, and both are the kind of thing a reproducibility claim has to be able to survive being asked about.

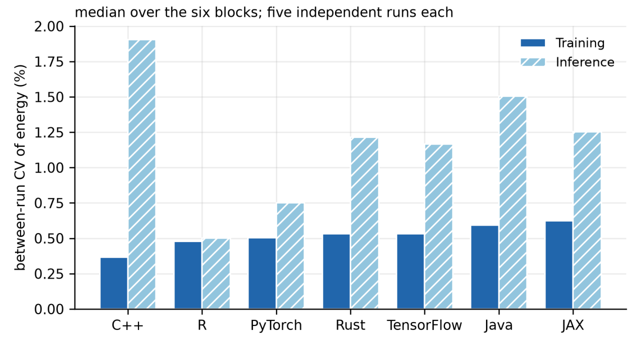

Figure 1: Between-run coefficient of variation of energy, median over the six (model, dataset) cells, from 5 independent runs per configuration.

6. Results

We report the campaign in the order the research questions were posed, after a subsection on repeatability that bounds everything after it. One departure is worth flagging: RQ4, on the apparatus, comes last but conditions everything before it, and a reader who wants to know how much to trust Sections 6.2–6.4 should read Section 6.5 first.

All figures below are between-run. Each configuration contributes 5 independent runs, and every interval, test and effect size is computed across those runs rather than across the epochs within one.

6.1. Repeatability of the measurements

Before comparing ecosystems it is worth asking how precisely the apparatus measures any single one of them, because the answer bounds every comparison that follows and because the submitted design could not produce it at all.

Across all 42 configurations, the median between-run coefficient of variation of training energy is 0.49 % and the worst is 1.21 %. Inference, whose blocks are several times shorter, is noisier but still tight: a median of 1.17 % and a worst case of 3.04 %. Figure 1 breaks this down by ecosystem.

Two things follow. First, differences of a few percent between ecosystems are resolvable, and the differences we report below are far larger than that. Second, the run-to-run variability is small enough that a single run per configuration would have produced a plausible-looking number — which is precisely why a single run per configuration is not evidence that the number is right. Low variance is a property of the measurement, not a guarantee about the thing measured.

6.2. RQ1: Impact of ecosystems on training energy efficiency

Figure 2 and Table 4 report training energy per epoch for every ecosystem and block, at a common board-and-package boundary.

The spread between the cheapest and the most expensive ecosystem ranges from 7.4× to 9.8× depending on the cell (Table 5), and which stacks occupy the two ends changes with the architecture: C++ is cheapest in every ResNet-18

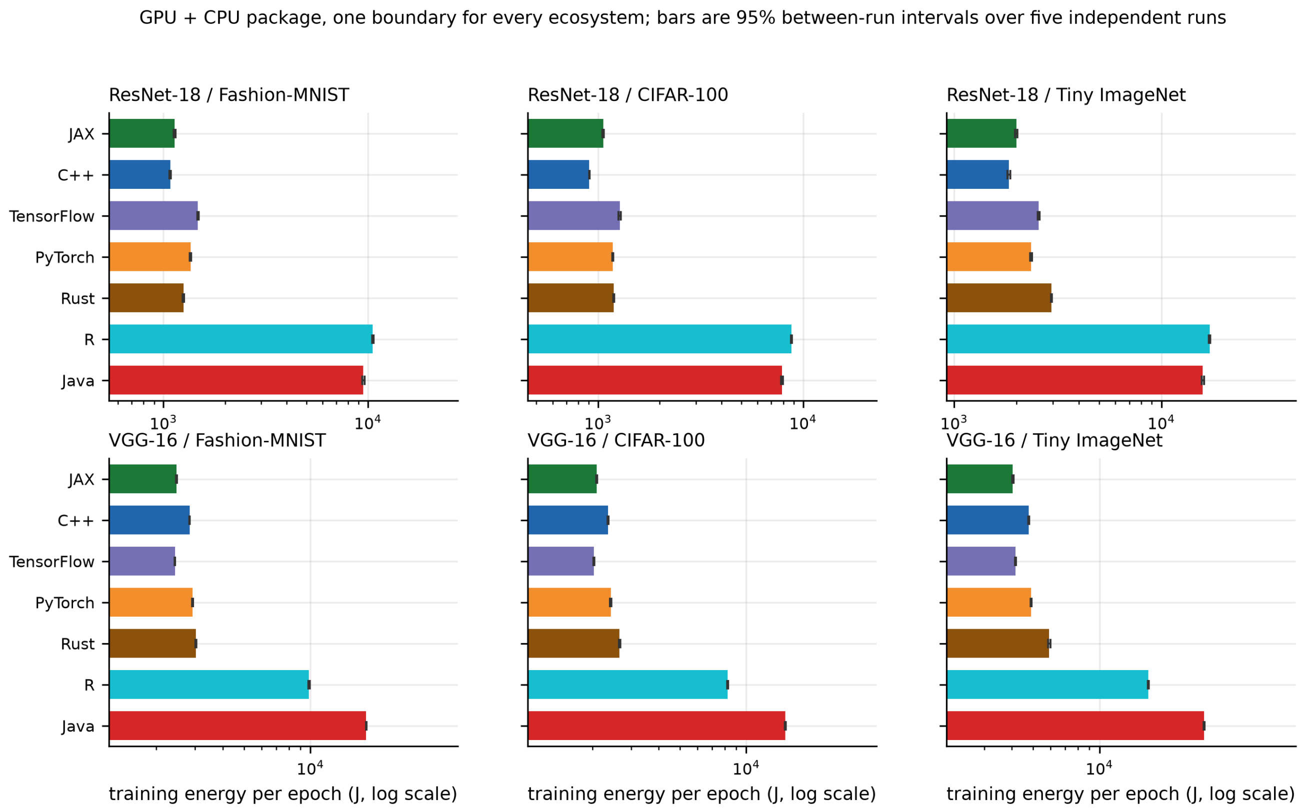

Figure 2: Training energy per epoch, accelerator plus CPU package, with 95% Student-t intervals over 5 independent runs. Note the logarithmic axis: the intervals are narrow enough to be hard to see at this scale.

Table 4
Mean training energy per epoch (J), accelerator plus CPU package.

Ecosystem	ResNet-18			VGG-16		
	CIFAR-100	F-MNIST	Tiny-IN	CIFAR-100	F-MNIST	Tiny-IN
C++	906	1 080	1 838	2 355	2 833	4 768
JAX	1 058	1 132	1 996	2 094	2 471	4 044
PyTorch	1 178	1 358	2 359	2 426	2 917	4 898
Rust	1 193	1 254	2 948	2 660	3 021	5 918
TensorFlow	1 274	1 474	2 559	2 033	2 433	4 164
Java	7 904	9 488	15 848	15 029	17 910	29 855
R	8 792	10 563	17 087	8 228	9 852	16 646

block, TensorFlow in two of the three VGG-16 blocks and JAX in the third, while the most expensive is R on ResNet-18 and Java on VGG-16.

The shared-module control group. 3 of the 7 ecosystems — Python/PyTorch, C++ and Rust — load the same exported TorchScript module on the same pinned LibTorch build under specification S1, and all 7 run under the single precision policy of Section 4.1. What the group controls is the model and the backend build: one canonical definition per architecture, exported once per (architecture, dataset) pair, with every stack asserting the manifest's parameter count at startup and the manifest recording a hash of the parameters in name order. It does not control everything else those three stacks do differently, and we no longer read it as though it did.

Table 5
Cheapest and most expensive ecosystem per block, training energy per epoch.

Model	Dataset	Lowest		Highest		
			J		J	Ratio
ResNet-18	CIFAR-100	C++	906	R	8 792	(9.7×)
ResNet-18	Fashion-MNIST	C++	1 080	R	10 563	(9.8×)
ResNet-18	Tiny ImageNet	C++	1 838	R	17 087	(9.3×)
VGG-16	CIFAR-100	TensorFlow	2 033	Java	15 029	(7.4×)
VGG-16	Fashion-MNIST	TensorFlow	2 433	Java	17 910	(7.4×)
VGG-16	Tiny ImageNet	JAX	4 044	Java	29 855	(7.4×)

Within the control the spread is 1.1× to 1.6× (Table 6), against 7.4× to 9.8× across all 7. On ResNet-18 the group spans 1.3×–1.6× and accounts for at most 21 % of the log spread; on VGG-16 it spans 1.1×–1.2× and for as little as 3 %. On both architectures, then, most of what separates ecosystems separates the stacks that do not share the module from the ones that do.

An earlier version of this study read a mechanism out of this table, and it was wrong. It argued that whatever separates those three cannot be the kernels, because the kernels are the same code, and it put about half of the ResNet-18 log spread under binding overhead, data movement and host-side dispatch. Sharing a build is not sharing a kernel configuration. Specification S4 asked for fp32 with TF32 disabled, and exactly one stack of 7 was given it, the other six taking cuDNN's Ampere default — so the two campaigns hold that ablation between them. Python/PyTorch's own ResNet-18 training energy is 3.14× higher in the campaign that denied it TF32 than in this one, on CIFAR-100 — the one cell the two campaigns leave comparable — while the 3 stacks of that cell whose precision policy did not change move by at most 1.3 %; an isolated kernel probe puts the same flag pair at 4.04× on the accelerator alone (Section 6.2.2). The PyTorch-versus-C++ gap the earlier campaign measured there, 3.94×, is 1.25× in this one, and it was nearly absent on VGG-16, whose classifier is GEMM rather than convolution — which no account of binding overhead predicts. Two further premises were false: the three stacks did not begin from identical parameters (Section 6.2.1), and the stacks outside the group were not all running the same network under the name VGG-16. All of that is fixed in the campaign reported here. What we report from Table 6 is the spread that survives holding the model and the backend build fixed. We do not decompose what is left, because a black-box design of this shape cannot, and the cost of guessing is on the record above.

R is reported beside the control rather than inside it. It links its own bundled LibTorch and, because the R binding cannot switch a script module between training and evaluation mode, builds an equivalent architecture instead of loading the shared one — so it shares neither the build nor the module. Including it widens the "shared-module" spread to 3.5×–9.8×, which would have been reported as binding overhead and is not: the difference between the two columns of Table 6 is the cost of one member that shares less than the label claims.

Energy at equal quality. A fixed-budget comparison rewards a stack for learning less. Because every ecosystem now records test accuracy per epoch, we can check whether it is doing so. Table 7 reports final test accuracy by ecosystem, architecture and dataset, with the per-block spread beneath it. On Fashion-MNIST all 7 stacks land within 0.5 percentage points of one another on either architecture, which is the strongest available evidence that they are running the same experiment — and a check the submitted design could not perform, because no ecosystem wrote its accuracy to disk. We quote the per-architecture figure deliberately: pooling ResNet-18 and VGG-16, which reach different accuracies in opposite per-ecosystem directions, would report 0.3 points and would be measuring the pooling rather than the stacks.

Table 6
Spread within the shared-module control group against the full spread, per block. The control holds the LibTorch build, the exported module and the data pipeline fixed, and the precision policy is fixed across all 7 stacks rather than only these 3. The LibTorch-family column adds R, which shares the lineage but neither the build nor the module.

Model	Dataset	All 7	Shared module & build (3)	LibTorch family (4)	Share of log spread
ResNet-18	CIFAR-100	9.7×	1.3×	9.7×	12%
ResNet-18	Fashion-MNIST	9.8×	1.3×	9.8×	10%
ResNet-18	Tiny ImageNet	9.3×	1.6×	9.3×	21%
VGG-16	CIFAR-100	7.4×	1.1×	3.5×	6%
VGG-16	Fashion-MNIST	7.4×	1.1×	3.5×	3%
VGG-16	Tiny ImageNet	7.4×	1.2×	3.5×	11%

Table 7
Final test accuracy (%) after the fixed epoch budget, per (architecture, dataset) block, mean over the 5 runs of each cell. The *Spread* row is the highest cell of a column minus the lowest, which is the per-block quantity the text quotes. No run is excluded from any cell: none collapsed to chance (Section 6.2.1).

	Fashion-MNIST		CIFAR-100		Tiny ImageNet	
Ecosystem	R-18	VGG-16	R-18	VGG-16	R-18	VGG-16
C++	90.26	92.65	29.51	33.01	14.09	14.89
Java	89.94	92.67	28.50	33.75	13.93	15.39
JAX	90.30	92.93	30.97	32.01	14.03	14.57
PyTorch	90.42	92.68	28.39	32.98	14.21	15.28
TensorFlow	90.31	92.74	28.23	33.90	13.92	14.52
R	90.45	92.51	34.73	34.65	17.22	14.68
Rust	90.02	92.60	28.88	33.74	14.01	15.09
Spread	0.5	0.4	6.5	2.6	3.3	0.9

On the harder datasets the stacks separate more — to at most 6.5 points per block, the widest cell of the same *Spread* row, with no run excluded because none collapsed (Section 6.2.1) — and there the energy comparison must be read alongside the accuracy reached rather than instead of it: on both harder datasets the stack that reaches the highest accuracy is also among the two most expensive to train (Table 4).

Figure 3 plots the two together. Ranking ecosystems by accuracy per kilojoule rather than by energy alone puts C++ ahead of Java by 7.9× — a different quantity from the energy ratio, and closer to the one a practitioner choosing a stack actually faces. That pooled figure averages values two orders of magnitude apart, since accuracy per kilojoule on Fashion-MNIST and on Tiny ImageNet are not the same kind of number, so we give it per dataset as well: 8.0× on Fashion-MNIST, 8.0× on CIFAR-100 and 7.7× on Tiny ImageNet, with JAX best on Fashion-MNIST, C++ on CIFAR-100 and C++ on Tiny ImageNet. The three agree with the pooled figure and with each other, so in this campaign the pooling does not distort the ratio; we report them separately because nothing guaranteed that in advance.

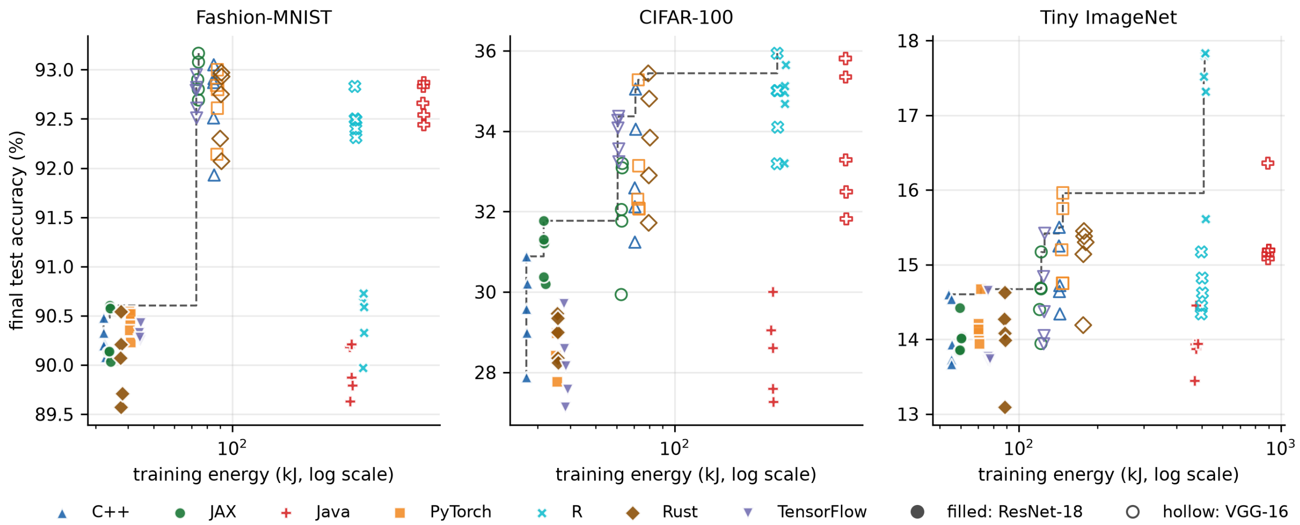

Figure 3: Energy spent against accuracy reached, one point per run. The dashed line is the empirical frontier: no configuration to its left reaches the same accuracy for less energy.

Energy to a target, and what it costs to report it. Closest of all is the energy a stack spends before it first reaches a stated accuracy, which a fixed-budget design cannot produce. Table 8 reports it against a target fixed per dataset. On Fashion-MNIST every run of every ecosystem reaches 85 %, and the energy to get there spans 6.6× — a like-for-like comparison at equal quality, and the cleanest number in this paper. On CIFAR-100 every run of every ecosystem reaches 30 % as well. Tiny ImageNet is where the table is informative for its gaps: 6 of the 7 stacks never reach 20 % within the epoch budget at all, so for those cells there is no energy-to-target to report and the fixed-budget columns are the only comparison available. We show the empty cells rather than dropping them, because "did not reach the target" is the result.

The one cell that is neither empty nor complete must not be compared with either kind. The single stack that does reach 20 % reaches it in some of its runs and not others, so its median is taken over the runs that succeeded — it conditions on success — and a median over survivors is not an estimate of the same quantity as a median over all 10. The attainment count is printed beside such a cell for that reason, and the like-for-like readings of this table are the Fashion-MNIST and CIFAR-100 columns, where every run of every stack reaches the target.

Table 8
Median training energy (kJ) to first reach a target accuracy, with the number of runs that reached it where that is not all 10. Targets: 85 %, 30 %, 20 %.

Ecosystem	Fashion-MNIST *85%*	CIFAR-100 *30%*	Tiny ImageNet *20%*
C++	3.4	14.0	never
Java	22.5	105.8	never
JAX	5.4	16.3	never
PyTorch	3.6	14.9	never
TensorFlow	4.3	14.4	never
R	14.7	49.8	86.4 (5/10)
Rust	3.6	20.2	never

6.2.1. Runs that consumed the budget and learned nothing, and why none did here

Replicating each configuration surfaced something a single run cannot show. In the first campaign of this design VGG-16 did not always train. We call a run *collapsed* when its final test accuracy never exceeds 1.5× chance for its dataset; the multiplier is generous on purpose, because every collapsed run sat at chance exactly — 1/100 on CIFAR-100, 1/200 on Tiny ImageNet — so no case depends on where the line is drawn. By that criterion 12 of 105 VGG-16 runs collapsed and stayed collapsed for all 30 epochs, having consumed the full energy budget while learning nothing. ResNet-18 never does this in either campaign — 0 of 105 runs in the campaign reported here — and neither architecture does it on Fashion-MNIST.

The per-epoch traces say precisely what those runs were. In every one of them the loss sat at $\ln N$ for the N classes of the dataset — 4.605 on CIFAR-100, 5.298 on Tiny ImageNet — to within 0.0007, for all thirty epochs. That is the loss of a *uniform* output distribution: the network is not confidently predicting one class, it is producing the same distribution for every input, and no gradient signal is reaching it. VGG-16 as shipped carries no batch normalisation, and a plain sixteen-layer network driven by Adam at 10^{-4} over a hundred or two hundred classes can start there and never leave.

What decides whether a run does so is the initial weights. The previous version of this section said the opposite, and the premise it rested on was false: we argued that the 3 stacks described as loading a byte-identical exported module began from identical parameters and still disagreed about which repetitions collapsed, so the deciding factor had to be the stochastic path. They did not begin from identical parameters. Python's PyTorch harness built torchvision's network fresh and loaded no module at all, and the C++

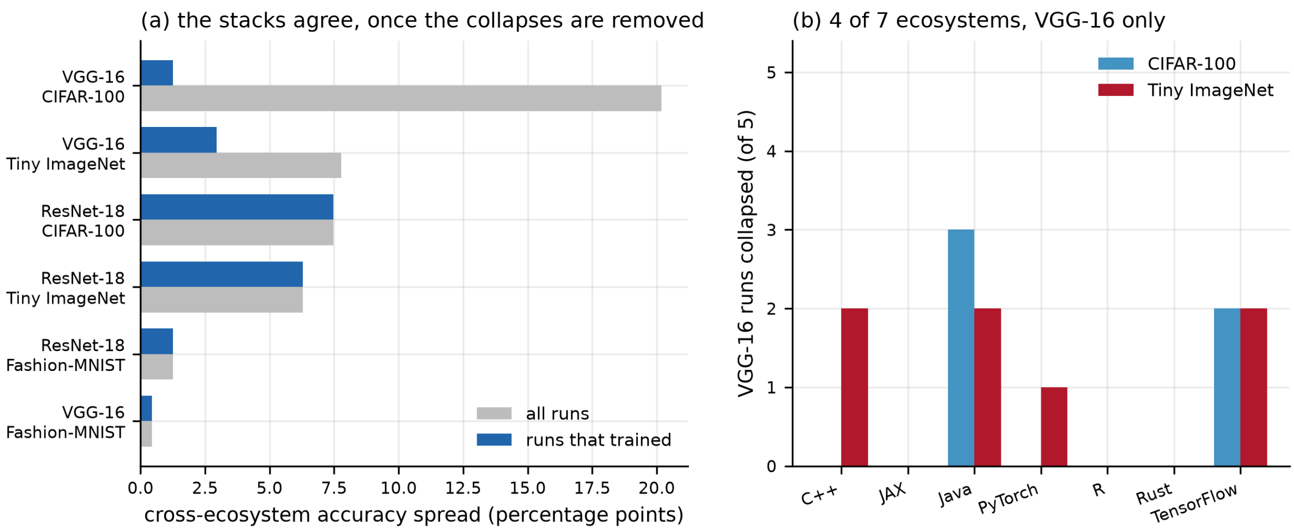

Figure 4: The superseded first campaign, from that campaign's own records. (a) Cross-ecosystem accuracy spread per block, over all runs and over only the runs that trained; the gap on VGG-16 is the collapsed runs. (b) VGG-16 runs that collapsed to chance, out of 5 per cell. The same figure for the campaign reported here is not drawn: with 0 collapses its panel (a) plots one set of runs against itself and its panel (b) is empty in every cell.

and Rust harnesses each loaded a module from a separate, unseeded export.

A controlled side experiment outside the campaign reverses the reading. VGG-16 on the many-class datasets, one framework, 6 runs per initialiser, with framework, optimiser, learning rate and data order held fixed and only the initialiser varied: 0 runs collapse under He initialisation, 2 under Glorot and 4 under Xavier. The stacks were not drawing from one distribution and diverging afterwards; they were drawing from different distributions. Deeplearning4j, the one stack with a hand-rolled initialiser — a truncated normal scaled by fan-in where torchvision scales, untruncated, by fan-out, 4.6× wider at ResNet-18's 7×7 stem — carried 5 of the 12 collapses. Those runs belong to neither campaign and no analysis script regenerates them; they are preserved as a record in the replication package. What we reported as a property of the ecosystems is a defect class about framework defaults: VGG-16 without batch normalisation under Adam at 10^{-4} is initialisation-critical, and frameworks ship different initialisers under the same architecture name. An architecture name does not fix the network that gets trained, and nothing in an energy table records which distribution a stack drew from.

The campaign reported here removes the cause rather than the symptom. Every stack starts from one canonical seeded export of each architecture, and the stacks that build their own layers use initialisers matched to torch's. Under that change VGG-16 collapsed 0 times in 105 runs. Zero events do not prove a zero rate: over the 70 runs in the cells where collapse ever occurred — VGG-16 on the two many-class datasets — the exact one-sided 95 % upper bound on the rate is 4.2 %, against the 17 % measured on the same cells before. And with no collapse in any ecosystem there is no table of rates to test: the exact test of homogeneity returns $p = 1.0000$ because every cell is zero, which makes it a statement about the table's shape rather than a test (Section 9 says why that test replaced the two we first ran).

A failed recipe and a broken pipeline look identical in an energy table. Both produce chance accuracy at full energy cost, and neither is visible in a Joule figure. The per-epoch traces separate them. In a genuine collapse the training loss does not move either, because a network that has settled on a uniform output has nothing left to fit. A pipeline defect looks the opposite: the training loss falls normally while test loss *rises* and accuracy stays at chance, which says that training and evaluation are not seeing the same inputs.

The diagnostic as we first built it could not do that for every stack, and we did not notice. It conjoins the two arms, and our Java harness recorded `NaN` for test loss in every one of its epoch rows; a comparison with `NaN` is false, so the conjunction could never fire for Deeplearning4j — the stack carrying 5 of the 12 collapses. We described that as a diagnostic degrading to its remaining arm. It did not degrade: Java's sensitivity to pipeline defects was zero rather than reduced, and no Java run was ever tested by it. Test loss is now computed in the same inference pass as accuracy, so both arms are live in all 7 stacks and Java's evaluation blocks no longer measure twice the work.

We report this because the discriminator found a defect of our own that the collapse count alone would have absorbed. 4 runs met the chance-accuracy criterion — at most 14.7 % — with training loss falling by 86–87 %: one Rust binary carried a private copy of the input transform, applied on the training path only, which resized the tensor its loader had already produced using a function that returns `uint8`. Every value in [0, 1] truncated to zero, so it trained on black images and was evaluated on real ones. Filed as collapses they would have read as "VGG-16 sometimes fails to train". Those runs were deleted and the configuration re-executed; since the evidence for a defect should be as traceable as the evidence for a result, and the discarded runs no longer exist in the campaign, their signature is preserved as a record in the replication package rather than quoted from memory.

The same defect class came back later in a different file, and what caught it was not this diagnostic. When the datasets were pre-resized offline so that each stack's own resize became a no-op, the `uint8` guard was written into one of the three Rust loaders and not the other two, so the Fashion-MNIST loader went on resampling an already 32×32 image and its pixel statistics stayed away from the other stacks' — not far enough to reach chance accuracy, and so invisible to a test that triggers on chance accuracy. It was found by comparing pixel statistics across all 7 stacks. All three loaders are guarded now. In the campaign as it stands the discriminator finds 0 pipeline defects and 0 collapses, which is what a campaign with nothing to separate looks like rather than a claim that the class is closed.

We are able to state the discriminator's recall on the one instance we have, and it is not perfect. The defect was a deterministic code path on the training branch, so all 5 repetitions of that configuration carried it; the discriminator flagged 4. The remaining run finished above the collapse threshold and so was never a candidate for the test at all, and the second instance of the same defect class — the loader described above — never came near the threshold either. That is precisely why a discriminator triggering on chance accuracy is a diagnostic aid and not a detector. One calibrated on a single positive example, with a threshold chosen after seeing it, is a heuristic: it separates the two failure modes cleanly once a run is known to have failed, and it will miss a defect that leaves enough accuracy behind. We report it as the diagnostic aid it is.

Two consequences matter here.

The stacks agree on what they compute. No block shows a cross-ecosystem accuracy spread above 6.5 percentage points, and the widest of them, on ResNet-18 on CIFAR-100, is a spread between runs that all trained. In the first campaign the same measurement had two arms — over all runs, and over the runs that trained — and almost the entire apparent disagreement between stacks sat in the gap between them (Figure 4a). There is no gap now: every run in this campaign trained, the two arms coincide, and the spread is what the stacks compute rather than what failed to train.

A collapse breaks fixed-budget energy comparison, invisibly. A collapsed run costs full energy and produces nothing: in the first campaign 10.7 % of the training energy went on runs that learned nothing, against 0.0 % of the 36.0 MJ spent here. An ecosystem whose default initialiser differed from the recipe's looks equally expensive and much less accurate; one whose default matched it looks efficient. With a single run per configuration — the design this study replaces — the reported number is whichever outcome that one configuration produced, and nothing in the energy data distinguishes the two cases. This is why we report accuracy alongside energy rather than assuming a fixed budget delivers fixed quality.

6.2.2. *The precision policy, and what it was worth*

This campaign was executed twice, and the difference between the two executions is a measurement. The first pinned TF32 off — `cudnn.allow_tf32` and `matmul.allow_tf32` false, `cudnn.deterministic` true — in the harness the Python stacks import. Only those three stacks import it, and of those only PyTorch reached cuDNN through it: TensorFlow was handed a mixed-precision policy, which is a different axis, and JAX was handed nothing. So one stack of 7 trained true-fp32 convolutions while the other six took cuDNN's Ampere default, which is TF32. The re-execution reported throughout this paper runs all 7 under the single policy of Section 4.1. Because the two executions differ in the precision policy of exactly one stack, over the same 42 configurations, they are between them an ablation of that policy against a control group whose precision policy did not change, and we report them as one. (The earlier campaign that motivated this design, in Section 8, is a different thing again and ran on different hardware; nothing here is compared against it.)

Table 9 puts the two executions side by side, per epoch of training, and says of every ResNet-18 training cell whether anything other than the precision policy changed between them. On CIFAR-100, where nothing else did, Python/PyTorch's ResNet-18 training energy is 3.14× higher with TF32 denied than with it allowed, while the 3 stacks that are comparable beside it — C++, Rust and R, whose precision policy did not change and whose remaining differences between the executions cannot move per-epoch energy — move by at most 1.3 %. And the quantity the previous version of this paper leaned on hardest, the Python/PyTorch-versus-C++ training gap on ResNet-18, falls from 3.94× to 1.25×. Each side of that comparison is 5 runs of thirty epochs, on real data, on the instrument this paper uses throughout.

Most cells are not a precision contrast, and the table says which and why. Three of the six stacks that kept their policy are excluded for reasons of their own: Python/TensorFlow ran a Model Garden ResNet-18 with a projection shortcut the other stacks do not have and dropped its last partial batch, Java/DL4J built its graph by hand with truncated convolution padding and a bias on every convolution, and Python/JAX was measured through a window that did not force completion in the first execution and does in this one. VGG-16 is excluded entirely, since the first execution ran four different networks under that name, so a VGG-16 ratio between the two executions is mostly architecture. Fashion-MNIST and Tiny ImageNet were re-encoded to the common 32×32 resolution on disk between the two, so on those datasets every stack's loader does different work; CIFAR-100 was already at that resolution and did not move. One caveat survives inside the cells that remain, and we state it rather than remove it: the first execution built PyTorch's ResNet-18 fresh from torchvision and this one loads the exported module of S1, so PyTorch's ratio contains whatever TorchScript execution is separately worth. The control group cannot separate the two, because the other stacks loaded a module in both executions. The ratio therefore bounds the precision effect from above, and we do not partition it. The

Table 9
Mean GPU energy per training epoch, ResNet-18, in the superseded campaign (TF32 denied for Python/PyTorch alone) against the current one (TF32 allowed for all seven). Five repetitions and 150 epochs per cell on each side. **Python/PyTorch on CIFAR-100 is the contrast the text cites**; the cells above the rule are the ones the two executions leave comparable, and each cell below it is excluded for the stated reason rather than for its value.

Ecosystem	Dataset	TF32 off (J)	TF32 on (J)	Ratio	Comparable
C++	CIFAR-100	741	742	1.00×	yes
PyTorch	CIFAR-100	2919	930	**3.14×**	yes[a]
R	CIFAR-100	6064	6147	0.99×	yes
Rust	CIFAR-100	975	968	1.01×	yes
Java	CIFAR-100	6611	6265	1.05×	network[b]
JAX	CIFAR-100	480	742	0.65×	window[c]
TensorFlow	CIFAR-100	777	980	0.79×	network[d]
C++	Fashion-MNIST	890	890	1.00×	data[e]
Java	Fashion-MNIST	7937	7518	1.06×	network[b]
JAX	Fashion-MNIST	538	801	0.67×	window[c]
PyTorch	Fashion-MNIST	3509	1077	3.26×	data[e]
TensorFlow	Fashion-MNIST	927	1139	0.81×	network[d]
R	Fashion-MNIST	8154	7390	1.10×	data[e]
Rust	Fashion-MNIST	1084	1027	1.06×	data[e]
C++	Tiny ImageNet	1703	1511	1.13×	data[f]
Java	Tiny ImageNet	13354	12566	1.06×	network[b]
JAX	Tiny ImageNet	1354	1414	0.96×	window[c]
PyTorch	Tiny ImageNet	5874	1871	3.14×	data[f]
TensorFlow	Tiny ImageNet	1884	1980	0.95×	network[d]
R	Tiny ImageNet	12630	11961	1.06×	data[f]
Rust	Tiny ImageNet	3379	2367	1.43×	data[f]

[a] PyTorch's harness also moved from eager torchvision to the exported TorchScript module between the executions, so this ratio bounds the precision effect from above.
[b] Java/DL4J built its graph by hand, with truncated convolution padding and a bias on every convolution.
[c] Python/JAX was measured through a window that did not force completion in the first execution and does in this one.
[d] Python/TensorFlow ran a Model Garden ResNet-18 with a projection shortcut the other stacks do not have, and dropped its last partial batch.
[e] Fashion-MNIST training images were re-encoded 28 → 32 on disk between the executions, so every stack's loader does different work.
[f] Tiny ImageNet training images were re-encoded 64 → 32 on disk between the executions, so every stack's loader does different work.

kernel probe below reports a larger ratio for the same flag pair, and the two are not in tension: the probe measures accelerator energy per step on a resident batch, where this figure is board-and-package per epoch and carries the loader and the CPU package with it.

Underneath that contrast we ran a kernel-level probe on an idle host (`scripts/probe_tf32.py`): ResNet-18 built from the definition the campaign exports, batch 128, one resident batch of synthetic input, and the flags set directly. Synthetic input is the point rather than a compromise, since which convolution algorithm cuDNN selects does not depend on what the pixels are, and holding one batch resident takes the loader out of the comparison identically in every cell. Every ratio here is against the configuration the other six stacks ran in the first execution: TF32 allowed to cuDNN, determinism unpinned, the matrix-multiply path at its framework default of fp32 — the cuDNN-disabled cell excepted, which is taken against this campaign's own default, the only configuration whose matrix-multiply flag it shares. A thousand training steps per measured window (three hundred for the cuDNN-disabled cell) clears the 5 s floor the script enforces against an NVML energy register that advances about ten times a second on this device (Section 6.5), so these ratios are not quantisation-bound; the cells are reshuffled from a recorded seed within each repeat, so drift over the run cannot alias onto one of them, and we report medians over the repeats. Denying TF32 to the convolutions costs 3.62× the accelerator energy per step. Denying it and pinning `cudnn.deterministic` as well — the first execution's configuration for PyTorch, exactly — costs 4.04×. Determinism on its own costs 0.99×, which is to say nothing we can measure; it is pinned nowhere in this campaign, and it is not expressible in the Deeplearning4j or R bindings at all. Allowing TF32 in the matrix-multiply path on top of the convolution path changes nothing we can measure on ResNet-18 either, because the network's only general matrix multiply is the 512 → 100 classifier head. One further cell disables cuDNN altogether, to bound what Deeplearning4j's absent cuDNN path can cost; that belongs with the cost of the Java stack rather than here. And the probe diverges from the campaign in one respect, which we state because it caps what the probe can be used for: it runs the eager module rather than the TorchScript export the campaign executes, so it measures cuDNN algorithm selection under the eager path.

That the matrix-multiply path is where ResNet-18's arithmetic does *not* go is also why the first execution showed nothing on VGG-16. Python/PyTorch, C++ and Rust all ran torchvision's VGG-16 head in that execution, so a comparison between them within it is like-for-like even though a comparison of VGG-16 across the two executions is not; and there the PyTorch-versus-C++ gap sat close to one, against 3.94× on ResNet-18. VGG-16's head is GEMM-dominated and the matrix-multiply path ran fp32 in every binding, so the gap disappeared exactly where the workload stopped being convolution.

What this costs the paper is a claim it made. The previous version measured the ResNet-18 gap between Python/PyTorch and the two stacks sharing its backend, found it large, and called it binding overhead. It was one configuration flag, in exactly one stack of 7, and we would not have found it without a control group of stacks that could not import the harness the flag lived in. The correction changes the finding's category rather than removing it: what was an undeclared confound is a declared ablation, and it says that a precision policy was worth more here than any difference between bindings this study measures. A cross-ecosystem energy comparison that does not state its precision policy is therefore not comparing ecosystems, whatever else it holds fixed. What remains of the Python/PyTorch-versus-C++ gap once the policy is common to all 7 is 1.25×, and we offer no account of it here.

Table 10
Mean inference energy per evaluation pass (J), accelerator plus CPU package.

Ecosystem	ResNet-18			VGG-16		
	CIFAR-100	F-MNIST	Tiny-IN	CIFAR-100	F-MNIST	Tiny-IN
C++	72	58	74	113	111	115
Rust	100	72	171	187	154	263
TensorFlow	160	154	164	211	210	234
JAX	165	154	166	262	262	268
PyTorch	181	173	184	238	239	276
Java	1 081	1 094	1 076	2 600	2 593	2 713
R	2 204	2 253	2 341	2 123	2 144	2 279

RQ1 Findings: Impact of Ecosystems on Training Energy Efficiency

Measured at a common board-and-package boundary and replicated 5 times per configuration, training energy differs across ecosystems by 7.4× to 9.8× depending on architecture and dataset, with C++ cheapest in every ResNet-18 block and the cheapest VGG-16 stack changing with the dataset. Between-run variability is small (median CV 0.49 %), so these differences are well resolved. On Fashion-MNIST final accuracy converges to within 0.5 percentage points on either architecture, so there the differences are energetic rather than qualitative. On the harder datasets the stacks do *not* reach the same accuracy — up to 6.5 points on a single (architecture, dataset) cell, with every run counted — and there the energy comparison must be read alongside the accuracy reached. Among the 3 stacks that load the same exported module on the same pinned backend build the spread is 1.1×–1.6× against 9.8× across all 7, so most of the effect is between stacks that do not share a model and a backend. One cuDNN flag was worth more than any of it: denying TF32 to a single stack cost 3.14× its ResNet-18 training energy, which is why a cross-ecosystem comparison that does not state its precision policy is not comparing ecosystems.

6.3. RQ2: Impact of ecosystems on inference energy efficiency

Table 10 reports inference energy per epoch over the test set, at the same boundary.

The inference spread is wider than the training spread (23.1× to 38.7×), and this is where the apparatus matters most. An inference pass at these dataset sizes takes well under a second in the fastest stacks, and blocks that short almost always fall in one of the padded modes characterised in Section 6.4. Any inference comparison drawn from an estimator's own duration field is therefore comparing padding rather than phases for exactly the stacks that are fastest. Our figures are counter-bracketed, which removes that artefact but is not on its own enough: a bracketed window measures only the work that has finished inside it. In an earlier execution of this campaign JAX's inference blocks closed over dispatched work that had not completed, and JAX was the stack that execution reported as cheapest at inference; the blocks here force every asynchronous stack to completion before the window closes (Section 8). The point generalises: the inference phase is where a cross-ecosystem energy study is most exposed to its instrument, and it is also the phase practitioners deploy.

RQ2 Findings: Impact of Ecosystems on Inference Energy Efficiency

Inference energy spreads more widely across ecosystems than training energy (23.1×–38.7×), and does so on blocks short enough that the measurement instrument, not the ecosystem, becomes the dominant source of error in a naive design. Reported inference rankings should be treated as claims about the apparatus until the window over which energy was accumulated is stated.

6.4. RQ3: Energy–time relationship across ecosystems

The claim that "faster is not greener" is a claim about two measurements, and it is only as good as the weaker of them. We therefore recompute it on counter-bracketed durations — the interval between the two counter reads that bracket a phase — rather than on any duration reported by the estimator.

On that basis, energy and time rank the configurations almost identically: Spearman $\rho = 0.96$ for training and 0.92 for inference, with 7.1 % and 11.5 % of configuration pairs discordant respectively. Those coefficients are pooled over all 42 configurations, so they are partly a statement about workload size: a bigger dataset costs both more time and more energy whatever the stack. Within a cell — one architecture, one dataset, one phase, the 7 ecosystems — the same coefficient runs from 0.68 to 1.00 over the 12 cells, so the relation is not an artefact of pooling workloads of different size. Within this workload and platform, execution time is a good proxy for energy, and the interesting question is why an analysis of the same experimental shape can conclude otherwise.

The answer is in Figure 5. Panel (a) plots the duration CodeCarbon reports for each block against the counter-bracketed phase. The excess between them is not continuous: it is concentrated at a few values with wide gaps between them. 25 % of blocks carry an excess of 0.01 s — that is, none — and 75 % carry seconds, with a median of 4.57 s. That second group is two populations rather than one, a second

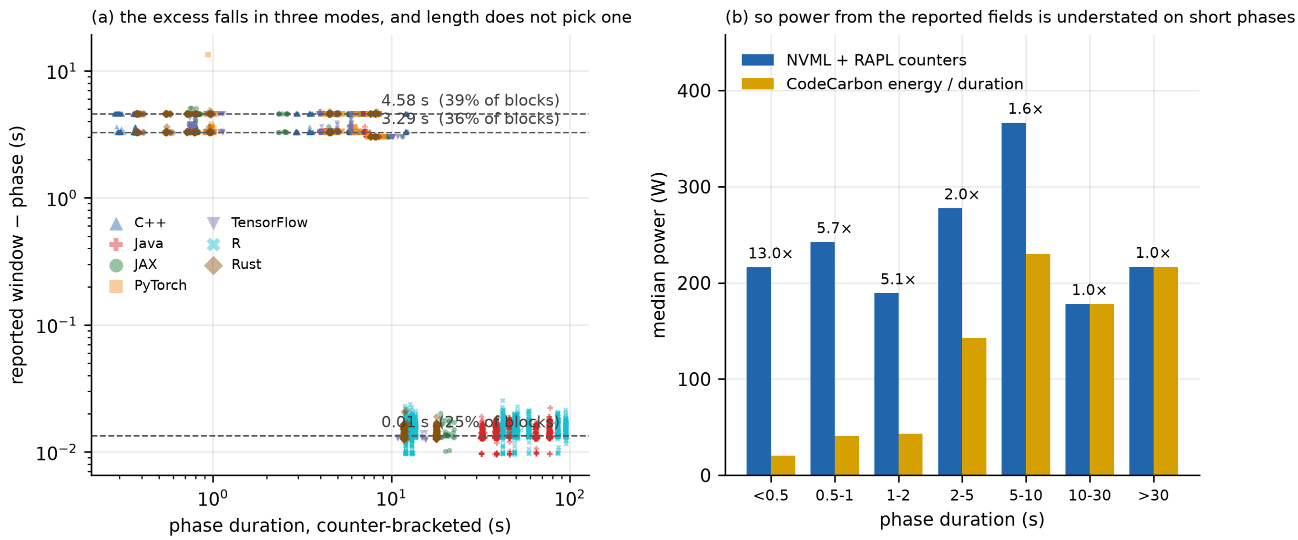

Figure 5: (a) The duration reported alongside each energy figure is not the interval the energy was accumulated over: the excess falls into 3 discrete modes, one per outstanding network lookup in `stop()`, with a threshold near 11 s. (b) Power obtained by dividing reported energy by reported duration is consequently understated for short phases and exact above ten seconds. Both instruments are compared on the terms both measure.

or so apart and 2.08 s across end to end; the blocks scattered between them are why the mode-finder we use, which cuts the sorted excess at gaps wider than half a second, reports the pair as one. 1 of the 12600 blocks sits far above both, at 13.38 s. 75 % of the campaign's blocks are therefore reported with a window seconds longer than the interval over which their energy was accumulated.

The cause is a network call, and it is reproducible in two minutes. `EmissionsTracker.stop()` resolves cloud metadata and then the host's geolocation over the network — two blocking HTTP requests — *after* the final energy reading. That is why only the duration is affected and the energy is not. Both results are cached on the tracker instance, and a background thread performs the same call once 8 measurements have accumulated. So a block pays for however many of the two lookups are still outstanding when it ends.

Table 11 is a direct probe on this host, with no accelerator and no workload: a tracker started, held open for a fixed interval, and stopped. Blocks that end before the background call pay 4.56 s; blocks that outlive one of the two lookups pay 3.01 s; blocks longer than 12 s pay 0.012 s. The campaign's excesses are not a mystery about the tracker's lifetime: they are how many network round trips remain, and all three of the probe's levels appear in the campaign. The padded blocks divide between the upper two, close to 3.01 s and 4.56 s, with every stack but R in both; which of the two a block falls in tracks the clock rather than the stack, as we report below. 1 block matches no level of the probe at all, and we flag it rather than explain it.

This explains the campaign. All 8942 blocks shorter than 9 s are padded, 100 % of them; the shortest unpadded block in the campaign is 10.8 s and the longest padded one 12.3 s, so the two populations meet only in that interval. A single threshold at 11 s classifies 98.8 % of the 12600 blocks correctly. It misses 154 — nearly all of them C++ training blocks, which are padded at durations where every other stack is not, and which we cannot account for — and it wrongly predicts padding for 3 blocks that sit just under the threshold.

Table 11
Cost of `EmissionsTracker.stop()` against the length of the block it closes, measured directly on this host with no workload (`scripts/probe_reported_window.py`). The three levels are two, one and zero outstanding network lookups.

Block held open	Cost of `stop()`	Lookups outstanding
2 s	4.581 s	2
5 s	4.560 s	2
8 s	4.559 s	2
9 s	3.014 s	1
10 s	3.014 s	1
11 s	3.016 s	1
12 s	0.012 s	0
15 s	0.012 s	0
20 s	0.013 s	0

Table 12 shows which stacks this reaches, and the answer follows from block length as the mechanism predicts. Every inference block in the campaign is under the threshold except R's, and every one of those short blocks is padded. Training divides the same way: Java and R, whose training epochs run for tens of seconds, are never padded; the stacks with short epochs almost always are. C++ is the exception in both directions, and we flag it rather than explain it.

Two consequences follow, and the second is the useful one. The magnitude of the padding is network latency on this host, not a constant of the tool: in our records the padded blocks divide by time of day, a median excess of 4.58 s in the night hours against 3.29 s in the day. And the whole artefact is a configuration away from disappearing — an offline configuration, or an `api_call_interval` of −1, removes

Table 12
Where the reported window is padded, by stack and phase, with the range of block lengths each stack produces. The pattern follows the threshold of Table 11; C++ training does not.

	Training		Inference	
Ecosystem	block length	padded	block length	padded
Java	31.6–79.8 s	0%	4.0–9.5 s	100%
R	41.6–95.7 s	0%	11.9–13.8 s	0%
Rust	4.4–18.2 s	67%	0.4–1.0 s	100%
PyTorch	4.2–12.7 s	83%	0.9–1.0 s	100%
JAX	4.0–22.5 s	97%	0.7–6.7 s	100%
TensorFlow	3.9–15.6 s	98%	0.7–1.2 s	100%
C++	2.9–12.3 s	100%	0.3–0.4 s	100%

the lookups. A study that measures short phases with this tool and does not set that parameter will record inflated durations, and nothing in the output says so.

We arrived at that explanation late, and the route is worth reporting because it is this paper's own argument applied to this paper. We first fitted a floor, max(phase, 4.94 s), and reported $R^2 = 0.9819$. That is wrong in the middle of the range, so we refitted two regimes — phase plus a constant below a threshold — and reported $R^2 = 0.9976$ at a third of the error. Then we found the three modes and the exceptions above the threshold, concluded that no length-based model could be right, and said so.

That last step was an over-correction. A curve fitted to a predictor can be right for the wrong reason, and it can also be right: what settles it is the mechanism, not the fit statistic, and until we measured `stop()` directly we had no mechanism and were arguing from residuals in both directions. The threshold is real, it has a cause, and the cause predicts where the threshold sits. The general lesson we would draw is narrower than the one we first drew: a high R^2 on a skewed predictor is not evidence of the right functional form, and neither is a scatter of exceptions evidence against one.

The consequence needs no model at all, which is the point. Table 13 divides each instrument's own energy by its own duration. Below half a second the estimator's fields give 20 W where the counters measure 216 W, a factor of 13.0. The distortion falls monotonically with block length and reaches unity above ten seconds, where the reported window is the accumulation window and the two agree exactly. Both sides of that comparison are the terms the two instruments both measure; putting CodeCarbon's total in the numerator instead would mix this artefact with the modelled RAM term criticised in Section 6.5, and did, in an earlier version of this table. The shortest phase in the campaign takes 0.28 s and is filed as 4.86 s, a factor of 17.

Panel (b) plots the same quantity per block. What matters about it is the *shape*: the bias is neither random nor uniform but monotone in phase length, so it falls hardest on the fastest ecosystems and on the inference phase, and it stretches their apparent time axis while leaving their energy untouched. That is precisely the shape of an artefactual "fast but energy-hungry" finding, and it is why a study can obtain that result from correct energy measurements.

Table 13
Power derived from the estimator's own energy and duration fields against power from the counters, by phase length. No model is fitted: each instrument is divided by itself, on the terms both of them measure. The last column is the median of the per-block ratios, not the quotient of the two medians beside it.

Phase length	Blocks	Power from the reported fields	Measured power	Understated by
<0.5 s	1050	20 W	216 W	13.03×
0.5–1 s	3376	40 W	243 W	5.69×
1–2 s	44	43 W	190 W	5.06×
2–5 s	2222	143 W	278 W	1.97×
5–10 s	2408	230 W	366 W	1.56×
10–30 s	1700	178 W	178 W	1.00×
>30 s	1800	217 W	217 W	1.00×

RQ3 Findings: Energy and Time

On counter-bracketed durations, energy and execution time rank the campaign's 42 configurations almost identically ($\rho = 0.96$ training, 0.92 inference): within this workload, faster *is* greener. The contrary finding is reproducible as an instrument artefact — the duration field of a widely used software estimator pads 75 % of blocks with seconds of tracker lifetime that carry no energy, inflating the apparent runtime of the shortest block by 17× while leaving its energy correct.

6.5. RQ4: How much of this is the apparatus?

Every block in the campaign carries two energy readings taken over the same phase boundaries. Figure 6 and Table 14 report what they say about each other.

On energy, the two readings agree — and it is important to say why. Over the whole campaign the ratio of CodeCarbon to counters is 1.0029 for the accelerator and 1.0053 for the CPU package, or 1.0034 for their sum, a mean per-block disagreement of 0.3 %; weighted by energy over the campaign it is 0.04 %.

We initially read that as an accuracy result and it is not one. Where both counters are exposed, CodeCarbon reads *the same registers we do*: NVML's accumulated-energy register on the accelerator and the RAPL `energy_uj` files on the CPU package. The two are not independent instruments; they are two readers of one instrument. The evidence is in the residual itself. The accelerator terms differ by a median of 1.2 mJ — floating-point noise on the same integer register — and essentially all of the residual is the CPU term, a near-constant 0.48 J per block whose correlation with block duration is 0.02. That constant is the counter reads being nested immediately inside the tracker's own window rather than coinciding with it: at this machine's package power it corresponds to a window a few milliseconds wider, and it does not grow with the block.

This changes what the comparison is worth, and we think it makes it more useful rather than less. It does not certify

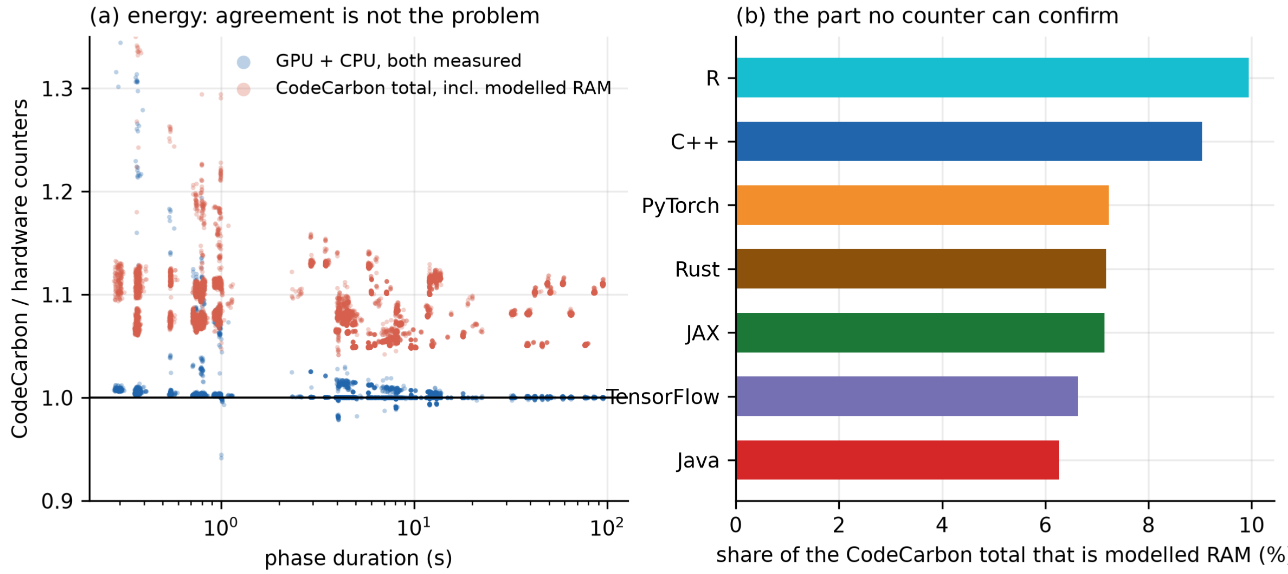

Figure 6: (a) Ratio of CodeCarbon to hardware counters, per block, against phase duration. The measured part agrees; the total does not, by exactly the modelled RAM term. (b) The share of each ecosystem's reported total that is modelled RAM.

Table 14
Agreement between the two readings over 12600 blocks. The mean is over per-block ratios and is dominated by the short blocks, which carry a fixed offset; the campaign-weighted column is the ratio of the two totals. The first three rows compare registers with themselves.

Quantity	Mean per block	5th–95th pct	Campaign-weighted
GPU, ratio (both read the NVML energy register)	1.003	1.000–1.006	1.000
CPU package, ratio (both read RAPL energy_uj)	1.005	1.000–1.022	1.001
GPU + CPU, ratio (the part both read)	1.003	1.000–1.008	1.000
CodeCarbon total over counters, ratio (incl. modelled RAM)	1.087	1.050–1.127	1.079
RAM share of the CodeCarbon total (per cent, not a ratio)	7.634	4.686–10.458	7.293

the estimator's accuracy against ground truth, which would need a wall meter and which independent validation work supplies [41]. What it certifies is that on a platform where the counters are exposed the estimator's energy is not a model at all — so the failure modes worth worrying about are not in the arithmetic but in the configurations where the counters are *not* exposed and the substitution is silent, and in the fields beside the energy. Both are what the rest of this section is about.

Two caveats belong with that. The first is that our reference is not arbitrarily fine either: NVML's accumulated-energy register advances about ten times a second on this device, so a block is quantised to roughly 100 ms of accumulation. At training power that is tens of joules per block, which is why the accelerator terms agree to a median of 1.2 mJ but not to that in the tail. It does not affect the run-level means this paper reports — the quantisation is unbiased and averages over thirty blocks — but a per-block agreement claim on a sub-second phase is at the resolution of the instrument, and the inference figures of Section 6.3 should be read with that in mind.

The second is that one figure we reported must be read carefully. Expressed as a percentage, a fixed 0.49 J offset grows without bound as blocks shorten: the mean per-block error is 0.01 % above 30 s and 0.9 % below one second. That is arithmetic on a constant, not a degradation of the instrument, and we withdraw the reading — which we published in an earlier draft of this analysis — that the transition marks the sampling interval. There is no transition: the absolute residual is flat across every duration decile of the campaign's blocks, from the shortest inference phase to the longest training one.

On what is not measured, they cannot agree. CodeCarbon's total exceeds the counters by 1.087×, and the excess is entirely its RAM term — 7.6 % of its reported total on average — which *in machine tracking mode* — the mode this campaign used, and the one a whole-machine study needs — is derived from installed memory capacity rather than from memory activity. The model is specific: a constant watts-per-DIMM, over a DIMM count *inferred* from total capacity, with decreasing marginal power above four DIMMs and a cap. In process mode it uses the process's resident set instead, so the criticism here is about machine mode and should not be read past it. On this machine it infers four modules from 30.2 GB and yields a flat 20 W for every block of the campaign, regardless of what the workload does to memory. No counter on this platform can confirm or refute it, because there is no DRAM RAPL domain to read. We could not read the machine's memory table to check the inferred count either — that needs privileges the measurement account does not have — and we note only that a desktop reporting this capacity is at least as likely to hold two modules as four, in which case the same model would charge half as much. The objection is not the factor but that the input is a guess about the hardware rather than a reading of it.

The number is not wrong so much as it is not a measurement, and it is added silently to a figure otherwise composed of measurements. On a machine with more installed RAM the model contributes more — bounded, because of the cap, but enough that the reported energy of an identical workload depends on the host's memory configuration. Our own boundary has the opposite failing and we state it plainly: the counters omit DRAM energy altogether. One figure contains an unverifiable term; the other is missing a real one. Neither is a whole-system number, and the difference between them is not an error bar.

The time between the phases belongs to the instrument. Energy is attributed only to the phases each stack brackets, and the stacks differ enormously in how much of a run that is. Between the start of a run's first block and the end of its last, tracked time ranges from 46 % (C++) to 100 % (R): more than half of a C++ run's measured interval falls outside any measured block. We state the denominator precisely because it is not the process's wall time — start-up, dataset construction, module loading, warm-up before the first epoch and teardown lie outside it, and are excluded from both sides of the ratio.

The natural reading is that the low-coverage stacks do a great deal of work between phases and are flattered by a per-phase comparison. We pursued that reading and it does not survive. The gap preceding each block is close to the amount by which CodeCarbon's reported window exceeds the counter-bracketed phase: the two correlate at $r = 0.98$ over every block in the campaign, they differ by a median of 0.26 s block by block, and the window excess accounts for 97 % of all untracked time. That pooled correlation is largely a contrast between padded and unpadded blocks rather than a fine agreement, and we report it as such: within the padded blocks alone it is $r = 0.85$, and within the unpadded blocks $r = 0.03$. The timestamps these gaps are computed from have 1 s resolution against a median gap of 3.51 s, and 9 % of the computed gaps come out negative before they are clipped at zero. That fraction, rather than the correlation, is the honest measure of what this attribution can resolve.

Coverage tracks block length rather than anything about the ecosystems (Table 15): short blocks pay the tracker's fixed cost more often, so the stacks with the shortest median block are penalised where those with the longest are not. The untracked time is the instrument holding its window open. It would not exist in an uninstrumented run, and charging it to the ecosystems would charge them for the cost of being measured. Per-phase energy is therefore the right quantity, and the spreads reported in Section 6.2 stand as they are.

Table 15
Tracked share of the interval from a run's first block to its last, and how much of the remainder is the instrument's own window. Not wall time: the denominator excludes start-up, dataset construction and teardown. Coverage follows block length, not the ecosystem — with R the exception, whose untracked time is almost none of it.

Ecosystem	Median block	Tracked share	Of the untracked rest, the instrument
C++	3.1 s	46%	101.1%
TensorFlow	3.2 s	48%	99.8%
JAX	5.0 s	49%	96.3%
PyTorch	3.9 s	54%	100.8%
Rust	4.3 s	59%	100.7%
Java	25.4 s	93%	98.8%
R	33.5 s	100%	14.3%

Two caveats belong with that conclusion. The attribution is not exact — for 3 of the 7 ecosystems the instrument's share of untracked time comes out slightly above 100 %, which is the one-second timestamp resolution showing through, and Table 15 prints it unclamped rather than rounding it away. And the untracked time is not free. Across the campaign it amounts to 10.5 hours. Pricing it is a range rather than a number: at this machine's measured idle draw of 64 W it is 2.4 MJ, about 6 % of the energy the campaign reports, and that is a floor — the interval begins microseconds after an epoch ends, with the accelerator not yet clocked down, so the true figure lies between that and the several times larger number the campaign's mean block power would give. Its share grows as blocks get shorter. It is also avoidable, now that we know what it is: it is not an intrinsic cost of machine-mode tracking but of leaving one configuration parameter at its default.

Does the instrument change the answer? We recomputed the ecosystem ranking of each block under both readings. They agree in 9 of 12 cells — 3 of 6 in training and 6 of 6 in inference — and the disagreements are adjacent-pair swaps, so no ecosystem moves more than one rank. We report both phases because an earlier version of this analysis reported the training figure alone, and in this campaign it is the weaker of the two: every disagreement is a VGG-16 training cell, while the inference cells — the phase whose sub-second blocks this paper treats as the fragile one — agree throughout. The substantive ordering of RQ1 is therefore unchanged — provided energy, and not power or time derived from energy, is the quantity being compared. Ranked by *derived power*, the two readings disagree everywhere, which is the finding of Section 6.4 restated.

RQ4 Findings: The Apparatus

Over 12600 doubly instrumented blocks, a widely used software estimator agrees with hardware energy counters to 0.3 % on the quantity both measure — because on this platform it is reading the same two registers, not estimating. That is the useful finding: where the counters are exposed, the estimator's energy is not the risk. The risks are the terms beside it. A modelled RAM term contributes 7.6 % of the reported total, which no counter can check. 75 % of blocks are reported with a duration seconds longer than the interval their energy was accumulated over, so power derived from those two fields is understated by up to 13.0×. Both produce plausible numbers; neither is visible from the estimator's output alone. The ecosystem ranking is nonetheless unchanged in 9 of 12 cells — 6 of 6 at inference and 3 of 6 in training, where the disagreements are adjacent-pair swaps.

7. Discussion

Our study highlights that the energy efficiency of DL workloads is not only a matter of algorithmic or hardware design, but is also materially influenced by the *choice of language–framework ecosystem* (i.e. the adopted stack, including frameworks/libraries, runtimes, and toolchains) [26, 27, 18, 21]. In this section, we interpret the results in light of their broader implications, and discuss lessons learned for different stakeholders.

7.1. Interpretation of Results

The ecosystem effect is large, and most of it is between stacks that share nothing but the specification. Under a common measurement boundary, the same architecture trained on the same data for the same number of epochs costs between 7.4× and 9.8× more energy on one ecosystem than another. 3 of the stacks in that comparison load the same exported TorchScript module on the same pinned LibTorch build, and the spread among them is 1.1×–1.6×: at most 21 % of the log spread on ResNet-18 and as little as 3 % on VGG-16 (Table 6). Fixing the model and the backend build removes most of the gap, on both architectures, and what remains sits between frameworks rather than inside one backend. We stop there deliberately: nothing in a black-box design of this shape decomposes the part that survives (Section 6.2.2).

Ecosystem-level overheads grow in absolute terms with workload scale. The absolute gap widens with dataset size and with model depth, so the practical relevance of the choice grows exactly where energy matters most. The mechanism cannot be isolated in a black-box study — runtime layers, input pipelines, execution engines and toolchains all differ — and we therefore report these as ecosystem-level effects rather than attributing them to a programming language.

The phases rank ecosystems similarly but not identically, and an estimator-based analysis exaggerates the difference. Recomputed on counter-bracketed energy, the same ecosystem is cheapest in both phases in 3 of 6 blocks, and the full ordering is identical in 0 of them, with Spearman correlations between each cell's training and inference energies of 0.54 to 0.93. This matters because the opposite claim — that the phases rank differently and so demand phase-aware stack selection — is a natural conclusion to draw from estimator output, and we can now say why: inference blocks in this workload are short enough that almost all of them fall in one of the padded modes of Section 6.4, so an inference ranking derived from an estimator's own duration is partly a ranking of padding. We do not claim phase-dependence never occurs; we claim that establishing it requires an instrument whose window is known.

Faster *is* greener, here. On measured durations, energy and time rank configurations at $\rho = 0.96$ (training) and 0.92 (inference). This contradicts a well-established finding in cross-language energy work [30, 32, 18], and we want to be careful about the scope of the contradiction. Those studies largely concern CPU-bound general-purpose benchmarks, where a language can trade instruction count against memory traffic and shift the energy-per-second substantially. In a GPU-bound DL workload the accelerator dominates and runs near a narrow band of power, so time and energy are nearly proportional almost by construction. The interesting result is not that we disagree, but that a study of this exact shape can produce either answer depending on which of two fields of the same CSV it treats as the duration.

Quality must be measured, not assumed. Under a fixed epoch budget the stacks converge to within 0.5 percentage points on Fashion-MNIST but separate by up to 6.5 points per block on the harder datasets. Any efficiency claim on those datasets that does not report accuracy alongside energy is comparing stacks that did different amounts of learning. This is not a subtle correction: an ecosystem that converges more slowly looks efficient precisely because it accomplished less. Here the corrected and uncorrected figures coincide — 4.0 points on CIFAR-100 pooled over both architectures, either way — because no run collapsed (Section 6.2.1). In the campaign this one replaces they did not coincide, and the difference between them was runs at chance accuracy averaged in as though they were measurements.

7.2. Implications for Stakeholders

Our findings carry different implications for different communities:

- **Researchers:** energy should be reported alongside accuracy, and both should be reported alongside the measurement boundary. Our stronger recommendation is procedural: a cross-stack study needs a written specification and a mechanism that enforces it. We found that every divergence we eventually corrected had produced a plausible number first, that inspection did not catch most of them and executable checks did, and that two were caught by neither — one by a power reading, one by the campaign refusing to take a step.

- **Practitioners:** ecosystem choice has a first-order effect on energy, and in this workload the cheapest stack is the same in both phases in 3 of 6 blocks with the full ordering identical in 0 of them, so a phase-specific stack split needs its own measurement rather than an assumption either way. Efficiency must nonetheless be weighed against maintainability, hiring, library availability and time to delivery; an energy gap of this size is not a mandate when the cheaper stack has a tenth of the ecosystem around it.

- **Framework designers:** efficiency stems from the ecosystem as a whole. The gap we measure lies mostly between stacks that do not share a model and a backend; holding those fixed removes most of it, and what remains this design does not resolve to a layer. So the target we can name is concrete rather than general, and it is the convolution backend: Deeplearning4j at its final release reaches no cuDNN path at all, and denying cuDNN to a stack that has it costs 13.07× the energy and 16.42× the time on ResNet-18 — an upper bound on what losing cuDNN can cost, not an estimate of Deeplearning4j's own penalty. We would also ask for one specific thing from measurement tooling: report the interval over which energy was accumulated, and fail rather than substitute a model when a counter is unavailable. An instrument that refuses to run is more useful than one that quietly estimates.

- **Policy makers and funding agencies:** AI efficiency is multi-dimensional, and we caution against adopting any single metric as a target (Goodhart's Law). Reporting requirements should ask for the measurement boundary and the replication count, not only the number: a figure without either is not auditable, and our own experience is that unauditable figures are wrong more often than not.

7.3. AI Requires Explicit Energy Measurement

Ecosystem selection is not a neutral engineering choice: it shifts DL energy consumption by a factor that is large and reproducible across repetitions of the same configuration. That is the substantive result. What it is not is a decomposition: we can say how much of the spread survives holding the model and the backend build fixed, and we do not claim to know what the remainder is made of.

How wide that factor is depends on where the measurement boundary is drawn, and the choice is not visible in the resulting number. The superseded first campaign shows this on its own records, without any comparison across machines — which would be improper, since the two campaigns ran on different hardware. Over those runs the ecosystem spread is 4.6× when energy is taken as the estimator's total and 8.5× when it is taken as the estimator's accelerator term alone. Both figures come from the same instrument; what changes between them is only which part of the machine is charged. A constant host term added to every stack compresses their ratios toward one, so widening the boundary narrows the apparent effect without changing anything that was measured. Any cross-ecosystem ratio is therefore a statement about a boundary as much as about the stacks, which is why we state ours and report it consistently.

The methodological result is that establishing it is harder than it looks. Every defect we found in building this study produced numbers of the right order of magnitude, in the right units, with plausible relative ordering. A learning rate that differs between two framework APIs, an evaluation loop written one image at a time, a linker dropping a CUDA dependency so that a binary runs on the CPU, an estimator reporting a padded window where a duration belongs — none of these announce themselves in the data. They are detectable only by a second instrument, an executable specification, or a quality metric that the study happened to record.

We therefore make the artefacts as important as the findings. The specification, the shared measurement contract, the conformance checker and the raw dual-instrument records are all in the replication package, and the manuscript's numbers are generated from them rather than transcribed. A reader who doubts a figure in this paper can regenerate it; a reader who doubts the apparatus can run the checks.

7.4. An Industrial Simulation of Deep Green AI Large-Scale Impact

To contextualize our empirical results in a hyperscale setting, we present an illustrative industrial scenario that translates our *relative* ecosystem-level energy differences into approximate energy and cost impacts. This scenario is not intended as a direct estimate for any specific production system, model family, or hardware stack; rather, it shows how small per-workload gaps can compound at scale.

Relative multipliers. For a given phase (training or inference) and ecosystem k, we define a phase-specific energy multiplier relative to any reference ecosystem *base* as:

$$m_k = \frac{\bar{E}_k}{\bar{E}_{\text{base}}},$$

where $\bar{E}_k$ is the mean energy measured under our experimental protocol for that phase. Accordingly, m_k should be interpreted as a *relative energy-per-work* factor under our setup, not as a universal constant.

Under the simplifying assumption that the ecosystem-level multiplier transfers to the same workload definition, for any baseline energy budget $\bar{E}_{\text{base}}$, the scaled energy is calculated as $\bar{E}_k = m_k \cdot \bar{E}_{\text{base}}$.

Training phase (illustrative scaling). From Table 4, moving from the most expensive ecosystem in a block to the cheapest changes training energy by 7.4×–9.8×. Taking a concrete case: if a *training* budget is $E = 1$ GWh on the most expensive stack of a block, the same fixed-budget run costs between 1/9.8 and 1/7.4 of that on the cheapest. At an electricity price of 100 e/MWh — a round figure in the range of recent European non-household prices, chosen for legibility rather than precision — that budget costs 100 000 € in

Table 16
Projected daily energy and cost for large-scale inference at a fixed service target (10^5 baseline accelerators, 24 h, electricity at 100 e/MWh). Fleet size scales with measured accelerator-hours; energy is that fleet at each stack's own measured draw. These are scenario arithmetic on measured quantities, not forecasts.

Ecosystem	Accelerator-hours	Fleet	Measured draw	Energy (MWh/day)	Cost (€/day)
C++	0.37×	36 796	197 W	174	17 365
Rust	0.68×	68 031	190 W	311	31 060
TensorFlow	0.81×	81 069	173 W	336	33 644
JAX	0.94×	94 170	166 W	376	37 560
PyTorch (baseline)	1.00×	100 000	167 W	401	40 088
R	13.22×	1 321 558	121 W	3 850	384 977
Java	6.06×	606 078	265 W	3 859	385 886

total, and the spread between the two ends of a single block is 86 416–89 778 € per GWh of training: not a margin on the bill but very nearly the whole of it. We stress that this is arithmetic on a relative multiplier, not a forecast: it assumes the multiplier transfers to a workload of a different size on different hardware, which is exactly the assumption our own measurements give us the least confidence in.

Inference phase (hyperscale workload, fixed work). Let PyTorch serve a fixed daily inference volume on N = 100,000 accelerators running continuously for 24 hours. We take the per-card draw from the campaign rather than assuming one: PyTorch's inference blocks measure 167 W at the accelerator, giving 401 MWh/day, or 40 088 € per day at 100 e/MWh.

Scaling that to another ecosystem takes two measured quantities, not one, and an earlier version of this scenario used only the first. At a fixed service target a less efficient stack needs more *accelerator-hours*, so the fleet scales with the time multiplier; and it draws its own power while doing so, which is not the baseline's. Both matter here, and they do not move together. Java needs 6.1× the accelerator-hours and draws 265 W per card against the baseline's 167 W, so the two compound and its energy multiplier is 9.63×, larger than either factor alone. They do not always compound: in Table 16 the ordering by fleet size is not the ordering by daily energy, because each fleet is multiplied by its own measured draw. Multiplying energy at a fixed fleet and a fixed per-card draw — as we did — would have asserted a draw for Java that its own measurements contradict, and one several times the board limit of the card.

Table 16 therefore gives fleet size, measured draw and the energy that follows. The multipliers are computed from accelerator energy alone, matching the per-accelerator quantity the scenario scales; at the board-and-package boundary the ecosystem spread is 24.6× rather than 22.2×. Note that the spread in accelerator-hours, 35.9×, is wider than the spread in energy: a fleet-sizing decision and an electricity bill are not the same comparison.

On that basis C++ serves the same volume at 0.43× the baseline energy and Java at 9.63×. The inference blocks these multipliers come from are sub-second in most stacks, which is the regime Section 6.5 identifies as least reliable for any instrument; they are counter-bracketed and so do not carry the estimator's padding, but they are the shortest measurements in the campaign and the ratios inherit that.

Takeaway. Ecosystem-level energy differences, treated purely as relative multipliers under a uniform protocol, translate into large operational deltas at hyperscale. Two caveats bound that statement, and both come from this study rather than from convention. The multipliers were measured on a single-accelerator desktop at 32 × 32 input resolution, and on inference blocks that are sub-second in the fastest stacks; nothing in this study establishes how they transfer to production model sizes, and we can bound neither the direction of that error nor its size. And the whole calculation is a scaling of *relative* differences: had we reported the modelled component of a software estimator's output as if it were measured, the same arithmetic would have produced a confident number from a quantity that depends on how much memory the host happens to have.

8. A catalogue of defect classes

Building this study meant first auditing an earlier campaign of the same design and then repairing every stack against the specification. Table 17 collects what that turned up. We report the defects as *classes* rather than as a list of bugs in one project, because each is a plausible mistake in any study of this shape, several are language or tool *defaults* rather than oversights, and most are invisible in the resulting measurements.

The last column is the one that matters. It asks whether a reader given only the released data — the tables, the CSVs, the plots — could detect the problem.

One row of that table deserves to be made concrete, because it is the class most likely to affect any study of this shape and the least likely to be noticed. Table 19 reports the training protocol *as implemented*, read from the source of an earlier campaign whose manuscript stated a single common learning rate.

Two stacks train at ten times the stated learning rate. Loader parallelism ranges from none to every hardware thread on the machine, and across the campaign it correlates with epoch duration at $\rho = -0.73$ ($p = 0.041$) over a 11.6× spread in duration and 9.9× in energy — which is to say that a column no one would think to report tracks the published ranking better than the programming language does. One stack normalises its inputs when no other does, so it is not even computing the same function. None of this is visible in an energy figure, and all of it is verifiable in about

Table 17
Defect classes of the instrument, with the measured effect of each; Table 18 continues the catalogue with the implementation, placement and analysis classes. "Visible" asks whether a reader given only the released measurements could detect the problem. Entries marked ∗ are ours: defects we introduced in the harness we built for this study. The unmarked entries were either found in the campaign this study audits or are framework and library defaults we had to override. What caught the marked ones differs by row — a conformance check, the accuracy metric, the second instrument, a parity probe, a power reading, and in one case only the campaign itself failing at its first batch — and the mixture is the point: no single mechanism found them all.

Class	Measured effect	Visible
Instrument		
Unit mislabelling	Energy reported in kW h under a J label; a factor of 3.6×10^6	yes
Reported duration is not the accumulation window	75 % of blocks padded by seconds of tracker lifetime in 3 discrete modes; power understated 13.0× on the shortest blocks	no
Modelled term inside a measured total	RAM energy from installed capacity in machine mode, 7.6 % of the reported total; scales with the host, not the workload	no
Mixed instrument versions	Versions 2.8.4 and 3.0.4 in one campaign: 231 W against 102 W of modelled host power for the same machine	no
Mixed tracking mode	1 of 8 stacks tracked per-process rather than machine-wide; its RAM term is 0.026 % of reported energy against 25 % to 55 % for the rest	no
Interpreter resolved from `PATH`	The instrument's version becomes a property of the shell; 5 of 8 stacks report a single constant CPU power, the signature of a modelled fallback	no
Unsynchronised tracker	Inference recorded as 0 J; training understated 24 %	no
Host not idle	Whole-machine tracking charges unrelated downloads to the workload	no
Two campaigns on one accelerator*	An orphaned driver restarted and wrote the same run directories while sharing the GPU: runs of 59, 60 and 43 epochs where the specification says 30, each measured against the other's contention	yes
Device work outlives the window*	A framework that dispatches asynchronously had its inference block closed before the device finished: a third of the work outside the window, and energy reported at 0.70× the figure the same block gives when the stack is forced to synchronise	no
A wrap guard with the wrong units*	The RAPL multi-wrap guard divided a counter range in ţJ by a power in W, which put its threshold six orders of magnitude out — 59 years rather than 31 minutes — and made it inert for every realisable block; it was also consulted only when a delta came back negative	no
Manifests without machine state*	No power limit, clock, persistence mode, governor, boost, driver version or precision setting recorded for any run, so nothing in the record establishes that two campaigns ran on the same machine in the same state	no

a page of code. The check we wrote to catch the learning-rate column passed on it, for the reason Section 4.1 gives: it asked whether some file matched, where the specification says every file must.

Three observations follow.

Only three of these are visible from the released data. One changes no conclusion, since a constant factor leaves every ranking intact. The second announces itself as an epoch count that does not match the protocol — visible, but only to a reader who thinks to check it against the specification, which is the whole reason for writing one down. The third is a test set silently shortened by sixteen images, and it announces itself only to a reader who multiplies a reported accuracy by the number of images it is supposed to be over. Everything else that changes a conclusion requires reading the code, or a second instrument, or a quality metric the study has to have thought to record.

Several are defaults rather than oversights. The linker's `-as-needed` behaviour, an estimator's fallback power constants, a framework resolving CUDA wheels that do not match its build, a resize function that returns `uint8` because the images it was written for were `uint8`. None is a lapse in care. A study can be executed carefully and land on all of them.

Some of them are ours. 12 of the 34 entries were introduced while building or repairing this study. Most were caught by a mechanism rather than by attention: a conformance check, the accuracy metric, the second instrument, a parity fingerprint, or the campaign driver refusing to start. Two were not. One was found by reading an accelerator that was drawing idle power, after a day in which five plausible hypotheses were each measured and refuted; the other by the campaign itself failing on its first batch with every check green.

One of them deserves singling out, because it is not a mistake in the usual sense. Moving the input transform into the loader is required by S3. Switching evaluation to batched inference is required by S3 as well. Each change is correct, and each was reviewed. Together they left a private copy of the transform on the training path and removed it from the evaluation path, and the result trained a network on black images for thirty epochs while reporting a plausible falling

Table 18
Defect classes, continued: what the stacks compute, where they compute it, and how the measurements were aggregated. Columns and marks as in Table 17.

Class	Measured effect	Visible
Implementation		
Divergent hyperparameters	2 learning rates across 8 stacks	no
Divergent loader parallelism	0 to 96 threads; $\rho = -0.73$ against epoch duration ($p = 0.041$) over a $11.6\times$ spread	no
Divergent input shape	One stack at $0.26\times$ another's `conv1` input volume; $0.07\times$ the energy	no
Divergent evaluation batching	Batch 1 against 128; 1.5–$1.95\times$ inference energy	no
Evaluation in training mode	Test accuracy $21\,\%$ against $86\,\%$; a different computation measured	no[†]
Resize discards a dtype conversion*	Image scaled to $[0,1]$, then resized by a function returning `uint8`; the scaling is silently thrown away	no[‡]
Two correct changes that are wrong together*	Moving the transform into the loader, and batching evaluation, each right alone; together they left a private copy of the transform on the training path only, which trained the network on black images for thirty epochs	no[†]
Channel order assumed, not checked*	`[C,H,W]` treated as `[H,W,C]`: the width cropped to three pixels and the result transposed	no[‡]
Ambiguous dataset layout	A class name containing / yields three different datasets	no
A precision flag set on one axis in one stack*	One stack of 7 pinned true-fp32 convolutions, one was handed a mixed-precision policy, one was handed nothing and four took the driver's default; 3.62× the accelerator energy per step separates the two branches, and no column of the metric files distinguishes them	no
Four networks under one name*	VGG-16 built independently in four frameworks spanned an order of magnitude in parameters, under a specification clause stating that counts were checked against the exported module and a checker in which no such check existed	no
Framework default initialisers diverge	Two frameworks' default distributions were $4.6\times$ and $3.3\times$ wider than the reference at the stem of the same architecture: the same graph, built from a different distribution, which a parameter count cannot see	no[†]
`momentum` means opposite things in three libraries	Batch-normalisation momentum is a retention factor in one framework and an update factor in another, and a third defaults to 0.99 with $\epsilon = 10^{-3}$: nine points of test accuracy on one stack, in the denominator of the efficiency metric	no[†]
Seeds that no stack used*	3 stacks of 7 were handed a distinct seed per repetition and ignored it — one hardcoded, one never forwarded, one rebuilt inside the epoch loop — while the conformance check reported five distinct seeds, having asked the campaign planner rather than the stacks	no
`drop_remainder` silently changes the test set	Two stacks took fewer gradient steps per epoch and were scored on 9,984 of 10,000 test images, always the same 16; detectable in the released accuracies, since test_acc$\times N/100$ is an integer only for the true N	yes
A resampling filter's default	4 stacks of 7 decoded different pixels on the one dataset that has to be downsampled; means agreeing while variances did not, over a third of the campaign	no
A normalisation applied twice*	One model in one stack divided by 255 an array its loader already returned in $[0,1]$, so it trained on $[0,1/255]$ under a clause fixing the pipeline for every stack; the batch normalisation after the first convolution absorbed it and the accuracies never showed it	no
Placement		
Silent CPU fallback	Whole workload on the CPU after a CUDA wheel mismatch; one warning line	no
Linker drops the CUDA dependency	Same source, same LibTorch: `Cpu` against `Cuda(0)`	no
Analysis		
Collapsed runs counted as measurements	12 first-campaign runs at chance accuracy averaged in with runs that trained, and the rate read as an ecosystem property rather than as a framework initialiser default	no[†]
Partial runs averaged as measurements*	A crashed run's fragment, $16\,\mathrm{J}$ against a neighbour's $300\,\mathrm{kJ}$, produced ecosystem spreads of 2×10^4	no
Epochs treated as replications	Effective sample size 1 per configuration; intervals far too narrow	no

[†] invisible because no stack wrote its accuracy to disk; visible the moment one does.
[‡] invisible in energy data; here it happened to abort the run, but the channel-order defect would have trained quietly on a three-pixel sliver had the dtype defect not stopped it first.

Table 19
Training protocol as implemented in the campaign submitted with the previous version of this paper, read from source rather than from the manuscript. The manuscript stated one learning rate and one input scaling; the code contained two of each. Divergences in bold. Specification S2 and S3 now fix every column; the conformance checks verify the learning rate against the source for every stack and the input scaling for one stack only, and the remaining columns are asserted by the specification and were checked by reading.

Ecosystem	Optimiser	LR	Loader mechanism	Threads	Input scaling
Python/PyTorch	Adam	1e-4	DataLoader workers	2	$[0, 1]$
Python/TensorFlow	Adam	**1e-3**	Keras generator	1	$[0, 1]$
Python/JAX	Adam	1e-4	Keras generator	1	$[0, 1]$
C++/LibTorch	Adam	1e-4	DataLoader workers	2	$[0, 1]$
Java/DL4J	Adam	1e-4	AsyncDataSetIterator	2	$[0, 1]$
R/torch	Adam	1e-4	dataloader workers	**0**	$[0, 1]$
Rust/tch	Adam	**1e-3**	rayon, all cores	**96**	**mean/std**
MATLAB/DLT	adam	1e-4	augmentedImageDatastore	1	$[0, 1]$

loss. No amount of care applied to either change in isolation would have caught it; what caught it was a metric neither change was about. We report them because the alternative — a paper claiming its authors made no mistakes — would undercut the argument it is making. The argument is that enforcement beats discipline, and the evidence for it is that our own discipline was not sufficient either. The qualification the round added is where the enforcement has to reach. An unenforced clause is a clause that will drift; a clause enforced against source rather than against the built binary is barely enforced at all; and a clause enforced against both still cannot tell you that the code takes a step.

9. Threats to Validity

As with any empirical study, our work is subject to threats to validity. We group them conventionally, and note where a threat that would ordinarily be speculative is one we can quantify from the data.

This campaign replaces one of our own, and we say what changed. An earlier campaign of exactly this design ran to completion and produced a coherent set of numbers. Auditing it against its own source afterwards found that three of its headline claims were confounded: one numerical-precision policy was in force for some stacks and not for others, the 7 stacks were training four different networks under the single name VGG-16, and the weight initialiser differed across stacks. We aligned the stacks — one precision policy, one canonical module per architecture, one initialiser — and re-executed all 210 runs. The same pass closed the asynchronous stack's measurement window, restored the test set two stacks had been scored on, brought three incompatible batch-normalisation conventions together, and threaded the seeds that three stacks had been handed and did not use. Every ecosystem comparison in this paper is measured after that alignment. Where the paper reports an earlier campaign of this design — the protocol read from its source in Table 19, and the defects it contributed to the catalogue — it is the campaign our previous version submitted, and it is named as such.

What the re-execution leaves standing is the apparatus, and it leaves the findings rather than the figures. The instrument comparison of Section 6.5, the padding and window-floor results and the boundary argument are statements about the tooling rather than about any one campaign's numbers: every quantity in them is re-derived from this campaign like everything else here, and several of them moved, but what they establish did not. The same holds for the source-level defects in the catalogue of Section 8: they were found by reading code, and re-running the code does not unfind them. The distinction is the one this paper asks its readers to make everywhere else — a result about what the stacks did needs the stacks to have been doing the same thing, and a result about the instrument does not.

Internal Validity. Whole-machine energy tracking means any concurrent work contaminates a measurement. The host was otherwise idle for the duration of the campaign; the blocks measured during development while a package download was running were discarded rather than corrected, and the protocol amended to forbid it. We report this because it is an easy mistake to make and an invisible one to inherit.

Toolchain heterogeneity remains. CUDA versions across the 7 stacks range from 11.6 (Deeplearning4j, whose final release predates CUDA 12) to 12.8 (the pinned LibTorch shared by the control group). This is not fully controlled and cannot be: an ecosystem is partly defined by the toolchain it supports. What we have changed is that it is now *bounded*. The 3 stacks of the control group are pinned to one build, and the precision policy is set campaign-wide rather than per stack (Section 6.2.2), so Table 6 separates the spread that survives a common model and a common backend from the spread that does not. It does not say what the surviving spread is. Pinning a build is not by itself an isolation of any one cause, and a policy written into a specification is not the same thing as a policy every stack receives.

Implementation drift between stacks was the dominant internal threat in studies of this shape, and it is the one this design attacks directly. Under specification S1 the 3 stacks of the control group load the same exported module rather than independent reimplementations, every stack asserts that module's parameter count at startup, and all 7 agree shape for shape on both architectures; S2 and S3 fix the optimisation and the data pipeline; and 92 automated checks verify the source-level and artefact-level clauses against the source, the built binaries and the campaign's own metric files, with shape-for-shape parity checked separately by `scripts/verify_architecture_parity.py`. 12 defects we introduced while building or repairing this study are catalogued in Section 8; most were caught by a mechanism — a check, the accuracy metric, the second instrument, or the driver refusing to start — and two were caught by neither a check nor a review, which is why we do not claim the specification is complete. We claim only that a divergence outside it now has to survive an executable check, and that the campaign itself is the last check in the sequence.

We applied our own fallback diagnostic to this campaign. The defect catalogue names the lowest mean accelerator power among the stacks as the signature of a stack that silently ran on the host, and in this campaign one stack trips it: R draws 127 W at the accelerator against up to 258 W for the others, on a 350 W card. A reader applying our criterion would suspect the stack that produces our largest reported ratios.

The CPU term clears it. If R were running the workload on the host its package power would rise, and it does not: 53 W, inside the 50–68 W band every other stack occupies, and the accelerator assertion of S4 refuses a CPU device at startup. R is waiting on the accelerator, not replacing it — its epochs are the longest in the campaign at a low duty cycle. We state this because a diagnostic published in a paper should be run against that paper's own data, and because the exoneration is a measurement rather than an assurance.

We can say what that duty cycle is. The accelerator was sampled at 1 Hz alongside the campaign, and across the 157 of 210 runs the sampler covered, R's reads 4.5–5.0 % utilisation on ResNet-18 and 12.1–13.4 % on VGG-16, with 1271–3001 MiB of device memory held by the R interpreter: the card is attached, resident and computing, and idle most of the time. R runs the two loader workers the specification mandates, the same two every stack gets. The other signature is in the spread rather than the level — across its runs R's accelerator power stays between 126 W and 141 W whichever network it is training, where every other stack's moves several times as far with the network.

Accelerator placement was verified rather than assumed. Every harness asserts the device at startup and refuses to fall back to the CPU. This is not hypothetical: linking a Rust binary against a CUDA LibTorch produces, by default, a binary that runs entirely on the CPU, because `-as-needed` drops the `torch_cuda` dependency that no Rust symbol references. Nothing in the energy data distinguishes that case from a correct one except its magnitude, which is why the check that catches it reads the binary rather than the energy: `readelf -d` on every Rust binary we build, refusing any whose dynamic section names no CUDA library.

Construct Validity. Our construct is energy at a stated boundary, not "sustainability". We report accelerator-board plus CPU-package energy read from hardware counters, which excludes the power supply, fans, drives, chipset — and host DRAM, for which this platform exposes no counter. We had no wall meter and therefore do not report a whole-system figure, and a board-and-package number must not be compared with a wall-meter number from another study.

The counters are our reference and they are not exempt from the scrutiny we apply to the estimator. RAPL's package energy is itself a modelled quantity on some parts and its update interval is of order a millisecond [57]; NVML's accumulated-energy register rests on board telemetry averaged internally, so its temporal resolution is not unlimited either. That bounds what the sub-second comparison of Section 6.5 can establish: for the shortest blocks in this campaign, both instruments are near the edge of what they resolve, and the disagreement we report there should be read as a joint property of the pair rather than as a fault of one. It does not bear on the reported-duration finding, which concerns a field rather than a measurement.

Two things about the counters changed in this round and both belong here. The first is a defect of ours. The guard that detects the RAPL energy register wrapping compared a reading in microjoules with a figure in watts, so the interval it computed before a second wrap became possible was decades rather than the tens of minutes it intended, and the guard could not fire; it was also consulted only when a delta came back negative, which is not what two wraps inside one block produce. No reported number moves — the longest block in this campaign is 95.7 s, far short of one wrap — but the guard had been inert for every block we have ever measured, and a guard that cannot fire reads exactly like a guard that never had to.

The second is that a measurement is only as reproducible as the machine state it was taken in, and ours recorded none of it: no power limit, no clock policy, no persistence mode, no ECC state, no governor, no boost setting, no driver version, not even a timestamp. A replicator could see every version of every library and nothing at all about the card. Every run of this campaign records that state in its manifest, written by both output paths so that the four stacks which never import our bridge are described as fully as the three that do. Provenance is the part still short: the harness now writes the source revision and a hash of the measurement path into each manifest, and the runs analysed here predate that, so which code produced them is argued from commit dates against run timestamps rather than read off the artefact.

That recording is owed to an observation we cannot explain. Two reviewers noticed independently, in an earlier campaign of this design, a bimodal accelerator power state, and we can say nothing more about it than that: none of that campaign's manifests records any machine state, so the conditions it was observed under cannot be recovered from the artefact, and we offer no mechanism. What is different now is that every run of this campaign carries the state a replicator would need — persistence mode, power limit, clocks, governor, boost and driver version — so a recurrence would be attributable rather than merely reported.

A second construct limitation belongs here, because it bounds what the attribution in Section 7 can claim. Every figure we report is un-baselined: the counters accumulate whatever the silicon draws, the machine's static consumption included. Un-baselined is not whole-machine, and the two are worth keeping apart — the boundary is the accelerator board plus the CPU package, and what a wall meter would add to it is quantified above. We measured that directly rather than arguing about it. On an idle host at the same boundary this machine draws 64 W — 25 W at the accelerator and 39 W at the CPU package — which is 23 % of the mean power of a measured block.

The two halves of the boundary are affected very differently, and this is the part that matters for interpretation. The accelerator's idle draw is 11 % of its campaign mean, so the GPU term is mostly workload. The CPU package's is 68 %: of the 58 W mean package power, the great majority is the machine being on. That term accordingly carries almost no workload signal — its coefficient of variation across the whole campaign is 12 % against 35 % for the accelerator — while contributing 21 % of reported energy. That is one reason we do not locate the ecosystem spread in host-side work: this boundary carries no measured host-side energy term to locate it in, and neither accelerator idle time nor the control group of Table 6 substitutes for one. A study wanting to measure host-side work directly would need DRAM and platform counters this machine does not expose. We report un-baselined energy because that is what the counters give and what an operator pays for, and we state the baseline so a reader can subtract it.

The more interesting construct threat is one we can measure. A software estimator's output is a mixture of measured and modelled terms, and the mixture is not stated in the output. Over 12600 blocks, CodeCarbon agrees with the counters to 0.3 % on the terms both measure, while its total exceeds them by 1.087× — the difference being a RAM term derived from installed capacity, 7.6 % of the reported total here and necessarily larger on a large-memory host. A figure of that construction corresponds to no physical boundary, and its value depends on the host's memory configuration rather than on the workload.

The estimator's reported duration is a second construct threat, and the sharper of the two: it carries seconds of tracker lifetime in 3 discrete modes rather than being the accumulation window, so any power or energy–time quantity derived from it is biased — most heavily on the shortest phases, though block length predicts which mode a block lands in without determining it. We use counter-bracketed durations throughout for this reason.

External Validity. We studied two convolutional architectures and three image classification datasets at 32×32 resolution; results may not generalise to other model families, to transformers, or to higher-resolution workloads, where the balance between accelerator work and host-side work is different. Which way that moves the ranking we cannot say: it depends on where the spread we measure comes from, and establishing that is outside what a black-box comparison can do.

The workload also does not saturate the accelerator. At this resolution and these dataset sizes the GPU runs well below its board power limit for much of each epoch, so what the campaign compares is whole pipelines rather than saturated kernels. That bounds the generality of the absolute figures, in a direction we cannot name: whether a saturating workload compresses the ecosystem spread is not something this design can settle, and it is the follow-up we propose in Section 11.

The platform is a single-accelerator desktop rather than a datacentre node, which makes the host share of total energy smaller than it would be on a multi-socket server. And MATLAB, present in the campaign our previous version submitted, is out of scope here because it cannot be brought into conformance with the specification; readers should not read its absence as a result about MATLAB.

Conclusion Validity. The submitted analyses of studies of this shape — including an earlier one of our own — treat the epochs of a single run as repeated measurements. They are not independent: epochs within a run share an initialisation, a JIT outcome, an allocator state and a thermal trajectory, so the effective sample size per configuration is one, confidence intervals are far too narrow and significance is inflated. Every interval and test reported here is computed on 5 independent runs per configuration, and 206 of the 210 runs were interleaved rather than executed consecutively, so that machine-state drift is spread across conditions instead of aliasing onto one ecosystem. The exception is stated at the end of this subsection.

Five repetitions is a small sample for a rank test, and we report what that costs rather than working around it. A Kruskal–Wallis test rejects equality of ecosystems in all 12 test groups ($p \leq 1.2 \times 10^{-5}$, $\varepsilon^2 \geq 0.95$), and 99 % of the 252 pairwise comparisons carry a large Cliff's delta. That effect size is saturated too, and we say so rather than leaning on it: with a between-run coefficient of variation under half a per cent, five-run groups almost never overlap, so δ reaches ± 1 for a 1.1× difference exactly as it does for a 38.7× one. It records that the groups separate, not by how much. Yet 0 of those 252 pairs is significant after Holm correction — and cannot be, at any effect size. With five runs against five, the smallest attainable two-sided exact Mann–Whitney p is 0.0079, and Holm over the 21 comparisons within a group multiplies it to 0.167. The design has the power to establish that the ecosystems differ and not the power to order any particular pair; we therefore rest the ordering claims on effect sizes and intervals, and make no pairwise significance claim.

Between-run variability is low — a median coefficient of variation of 0.49 % for training energy — which makes the design well powered for the effect sizes we report. Low variance is a statement about the measurement rather than a guarantee that the measurement is of the right thing.

Where the sample is genuinely too small we say so rather than reach, and the collapse rates of Section 6.2.1 are where we did not. In the first campaign those rates differed across ecosystems; we tested that table with a χ^2 test and with a permutation test and reported both, because they disagreed. The disagreement was a property of the two statistics rather than evidence about the data. χ^2 wants five expected observations per cell and that table gave it 1.7; our permutation statistic was the *range* of per-ecosystem rates, which sees only the extremes and cannot tell 7 ecosystems at 0, 0, 0, 10, 20, 40 and 50 per cent from two at 0 and 50 with five sitting at the mean. What we got wrong was running either test at all. Neither was the right test for that table, and the table was measuring the initialiser rather than the ecosystem.

The table warranted an exact test. Freeman–Halton, the $r \times c$ generalisation of Fisher's exact test, conditions on both margins and sums the probability of every table no more likely than the one observed: a few thousand terms, no approximation and no seed. On the first campaign's table it returns $p = 0.0040$. On this campaign's it returns $p = 1.0000$, because no ecosystem collapsed a run and there is nothing left to be inhomogeneous — with zero events in every cell the question is not which ecosystems differ but how tightly the rate is bounded, which is the 4.2 % of Section 6.2.1 and not a significance test at all. We keep χ^2 in the analysis only as the contrast we argue against, guarded so that a zero expected frequency reports no value rather than stopping the pipeline.

Finally, 4 runs — 3 configurations, all of them JAX — were executed in a separate window 2 days after the rest of the campaign. They cover VGG-16/CIFAR-100, VGG-16/Fashion-MNIST, VGG-16/Tiny ImageNet. They are therefore not interleaved with the other ecosystems, and machine-state drift between the two windows is not controlled for them by the design.

They are replays, and what they replace is worth stating. The original runs failed early in the campaign, on their first training batch: the shared classifier head carries a dropout layer, and our JAX implementation called the network without the random-number stream that layer requires. The failure came after the tracker had opened a block and recorded a parameter count, and the count was correct, because dropout has no parameters. Every check in the suite had passed on that code. We fixed it before the remaining JAX VGG-16 runs executed, so every JAX VGG-16 run in the campaign comes from one source revision, and the runs that predated the fix were deleted and replayed rather than repaired in place.

We measured it rather than reasoning about it, because the reassurance the argument wants — that the between-run variability is small — bounds variability *within* a window and says nothing about drift across days. We re-executed one already-completed configuration (PyTorch, ResNet-18 on Fashion-MNIST) in a third window, on the same machine under the same idle conditions and with the same seeds, and compared it run for run with its originals. The comparison is refused outright unless both windows record the same precision policy, and unless the calibration postdates the campaign it calibrates: a calibration produced by an earlier harness measures that harness, not the machine, and the first one we had for this campaign was exactly that.

Training energy in the later window is 1.2 % lower, which is 1.6 standard deviations of the within-window spread — the same order as the noise the design already carries, and small against the between-ecosystem differences it could confound. Inference is 3.2 % lower, 4.0 standard deviations against a within-window coefficient of variation of 0.9 %. That is several times the within-window spread and we do not call it small: in absolute terms it is 5.6 J per block, on the sub-second inference blocks that are the noisiest measurement in the campaign.

We therefore treat the late-window training figures as comparable and the late-window inference figures as carrying a between-window uncertainty of the order of 3.2 %, which is small against the differences between ecosystems reported in Section 6.3 but is not noise. It is a bound obtained from one configuration, in a different ecosystem and on a different architecture from the 3 it is applied to.

10. Conclusion

This paper presented an empirical study of how *language–framework ecosystems* affect the energy efficiency of *Deep Learning* workloads, across 7 ecosystems, two convolutional architectures and three datasets, replicated 5 times per configuration and instrumented twice per measurement block.

Three findings stand out.

First, ecosystem selection substantially shifts operational efficiency: at a common board-and-package boundary, training the same model on the same data costs between 7.4× and 9.8× more energy on one ecosystem than another, with between-run variability of order 0.49 %. 3 of the ecosystems compared load the same exported TorchScript module on the same pinned backend build, and among those three the spread is 1.1×–1.6× against 7.4×–9.8× across all 7: most of the gap is between stacks that do not share a model and a backend, and we report that without a decomposition of the part that remains.

Second, on the easiest dataset the effect is energetic and not qualitative: all 7 stacks converge to within 0.5 percentage points on Fashion-MNIST under the same epoch budget, doing the same work at very different cost. That equivalence does not extend to the harder datasets, where the stacks differ by up to 6.5 points on a single cell, and where energy must therefore be read alongside accuracy rather than instead of it.

Third — and this is the finding we did not expect — the apparatus determines more of the answer than the ecosystems do, in one specific and correctable way. Over 12600 doubly instrumented blocks, a widely used software estimator agrees with hardware energy counters to 0.3 % on energy. But the duration it reports beside that energy is not the interval the energy was accumulated over: 75 % of blocks carry seconds of tracker lifetime in which no energy was drawn, so power derived from the two fields is understated by up to 13.0× on the shortest blocks. Recomputed on measured durations, energy and time rank ecosystems at $\rho = 0.96$: in this workload, faster *is* greener, and the widely reported contrary result is reproducible here as an instrument artefact.

We draw a procedural conclusion from that. A cross-ecosystem energy study needs a written specification, a mechanism that enforces it, and a second instrument — for the same reason that a performance benchmark needs a stated warm-up policy. Every defect we corrected in building this study produced a plausible number first. The specification, the shared measurement contract, the 92-check conformance checker and the raw dual-instrument records are part of the replication package, and every figure in this paper is generated from them.

11. Future Work

Several directions remain open.

First, the workload should be extended beyond CNNs, to transformers and multimodal architectures, and to input resolutions that saturate the accelerator. Our workload does not: the GPU runs well below its board power limit for much of each epoch, so what the campaign compares is whole pipelines — loading, dispatch and kernels together — and not kernels alone. Whether the ecosystem gap narrows as the accelerator saturates is the single most useful follow-up, and it is a measurement, not a speculation.

Second, testing on heterogeneous hardware (server-class accelerators, TPUs, embedded and edge devices) would establish how much of the ranking is a property of the stacks and how much of this host. The hardware–software stack should be treated as part of the ecosystem definition rather than as a nuisance parameter.

Third, a white-box layer complementing this black-box design would localise where the remaining spread originates. The candidates sit on both sides of the boundary — loader thread scheduling, host-to-device transfer, per-kernel launch overhead and allocator behaviour on one side; kernel selection, algorithm choice and precision policy on the other — and choosing between them is precisely what a black-box comparison cannot do. We can say that 3 stacks holding the model, the data and the backend build fixed still differ by 1.1×–1.6×; we cannot say what that difference is made of.

Fourth, the instrument characterisation invites replication. We report the reported-window padding of one estimator on one platform. Whether other estimators share it, whether it varies with sampling interval and tracking mode, and how many published energy–time results rest on it, are answerable questions, and the method — bracket every block with hardware counters and compare — is cheap.

Finally, a wall-meter reference would let the board-and-package boundary used here be related to whole-system energy, which is what an operator pays for. We report that boundary consistently and decline to extrapolate; closing that gap requires hardware we did not have.

12. Carbon Footprint of This Study

The campaign reported here consumed 40.4 MJ (11.2 kWh) at the accelerator and CPU package across 210 runs. At the grid carbon intensity CodeCarbon assigns to this location, that corresponds to a few kilograms of CO_2e — modest against industrial training, and reported here for the same reason we ask others to report it: an energy figure without a stated boundary and a stated scope is not auditable. The figure above is the energy inside the measured blocks. It excludes development, failed runs and the two earlier campaigns of this design, one of them on other hardware and at another boundary — including those would make it several times larger — and it excludes the machine's draw during the untracked time between blocks discussed in Section 6.5, a further 2.4 MJ. An honest campaign-level figure is therefore larger than the one we report and we do not pretend otherwise; what we report is the quantity the comparisons are made on.

CRediT Authorship Contribution Statement

Leonardo Pampaloni: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing — original draft, Visualization. **Marco Pagliocca:** Methodology, Software, Validation, Investigation, Data curation, Writing — original draft, Visualization. **Enrico Vicario:** Resources, Supervision, Funding acquisition, Writing — review & editing. **Roberto Verdecchia:** Conceptualization, Methodology, Validation, Writing — review & editing, Supervision, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

All data and code underlying this study are public. The replication package contains the implementations for all 7 ecosystems, the shared measurement bridge, the experiment specification and its conformance checker, the campaign driver, the per-block dual-instrument records for all 210 runs and 12600 blocks, and the analysis pipeline that turns those records into every table, figure and quoted number in this manuscript. A single command regenerates the manuscript from the raw records, so any number here can be traced to the measurement it came from.

The package is an output of that pipeline and is built at the end of it, so it cannot be older than the tree it flattens — which it had quietly been, for a fortnight, while part of the analysis read it as an input. It refuses to build from a campaign that is not complete, on two conditions rather than one: the campaign must have produced every run it planned, and every run directory present must be whole. A package that silently omits or silently includes runs is worse than one that will not build. A restore script rebuilds the original run tree from the package and diffs it file by file, so the claim that the package is a copy is itself checkable, and every field is carried as text, because parsing a float and writing it back is not a copy.

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

During the preparation of this work the authors used generative AI assistants (ChatGPT and Claude) in three distinct capacities, which we separate because they carry different weight.

For *writing*, the tools were used to improve the readability of the manuscript text. For *software engineering*, they were used to repair and refactor the experimental implementations, to write the shared measurement bridge, the conformance checker and the analysis pipeline, and to diagnose defects in the ecosystems' data loaders and build configurations. For *analysis*, they were used to write the scripts that produce the tables and figures; the scripts, not the assistants, produce the numbers, and the numbers are injected into this manuscript from those scripts rather than transcribed.

No experimental measurement was produced, adjusted or selected by an AI assistant. The authors reviewed and edited all generated content and code, verified the findings against the raw data, and take full responsibility for the content of the article.

References

[1] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.

[2] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, "Quantifying the carbon emissions of machine learning," *arXiv preprint arXiv:1910.09700*, 2019.

[3] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.

[4] J. Xu, W. Zhou, Z. Fu, H. Zhou, and L. Li, "A survey on green deep learning," *arXiv preprint arXiv:2111.05193*, 2021.

[5] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[6] A. S. Luccioni and A. Hernandez-Garcia, "Counting carbon: A survey of factors influencing the emissions of machine learning," *arXiv preprint arXiv:2302.08476*, 2023.

[7] L. H. Kaack, P. L. Donti, E. Strubell, G. Kamiya, F. Creutzig, and D. Rolnick, "Aligning artificial intelligence with climate change mitigation," *Nature Climate Change*, vol. 12, no. 6, pp. 518–527, 2022.

[8] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. R. So, M. Texier, and J. Dean, "The carbon footprint of machine learning training will plateau, then shrink," *Computer*, vol. 55, no. 7, pp. 18–28, 2022.

[9] A. S. Luccioni, S. Viguier, and A.-L. Ligozat, "Estimating the carbon footprint of BLOOM, a 176B parameter language model," *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023, arXiv:2211.02001.

[10] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden, J. Cordaro, G. Di Guglielmo, J. Duarte, S. Gibellini, V. Parekh, H. Tran, N. Tran, N. Wenxu, and X. Xuesong, "MLPerf Tiny benchmark," in *Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks*, 2021, arXiv:2106.07597.

[11] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020.

[12] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[13] R. Verdecchia, J. Sallou, and L. Cruz, "A systematic review of Green AI," *WIREs Data Mining and Knowledge Discovery*, vol. 13, no. 4, p. e1507, 2023.

[14] V. Mehlin, S. Schacht, and C. Lanquillon, "Towards energy-efficient deep learning: An overview of energy-efficient approaches along the deep learning lifecycle," *arXiv preprint arXiv:2303.01980*, 2023.

[15] C. E. Tripp, J. Perr-Sauer, J. Gafur, A. Nag, A. Purkayastha, S. Zisman, and E. A. Bensen, "Measuring the energy consumption and efficiency of deep neural networks: An empirical analysis and design recommendations," *arXiv preprint arXiv:2403.08151*, 2024.

[16] T. Yarally, L. Cruz, D. Feitosa, J. Sallou, and A. van Deursen, "Uncovering energy-efficient practices in deep learning training: Preliminary steps towards green AI," in *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2023, pp. 25–36, arXiv:2303.13972.

[17] L. Cruz, J. P. Fernandes, M. H. Kirkeby, S. Martínez-Fernández, J. Sallou, H. Anwar, J. Bogner, J. Castaño, F. Castor, D. Feitosa, A. González, P. Lago, H. Muccini, A. Oprescu, P. Rani, J. Saraiva, F. Sarro, R. Selvan, K. Vaidhyanathan, R. Verdecchia, and I. P. Yamshchikov, "Greening AI-enabled systems with software engineering: A research agenda for environmentally sustainable AI practices," *ACM SIGSOFT Software Engineering Notes*, 2025, arXiv:2506.01774.

[18] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Ranking programming languages by energy efficiency," *Science of Computer Programming*, vol. 205, p. 102609, 2021.

[19] S. Georgiou, M. Kechagia, and D. Spinellis, "Analyzing programming languages' energy consumption: An empirical study," in *Proceedings of the 21st Pan-Hellenic Conference on Informatics (PCI)*, 2017.

[20] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. P. Fernandes, J. Saraiva, and J. Cunha, "Energy efficiency across programming languages: how do energy, time, and memory relate?" in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, 2017, pp. 256–267.

[21] A. Gordillo, C. Calero, M. Á. Moraga, F. García, J. P. Fernandes, R. Abreu, and J. Saraiva, "Programming languages ranking based on energy measurements," *Software Quality Journal*, vol. 32, no. 4, pp. 1539–1580, 2024.

[22] T. B. Chandra, P. Verma, and A. K. Dwivedi, "Impact of programming languages on energy consumption for sorting algorithms," in *Software Engineering*, ser. Advances in Intelligent Systems and Computing, vol. 731. Springer, Singapore, 2019.

[23] S. Mahadevappa and S. Figueira, "Energy-efficient programming languages for mobile applications," in *2021 IEEE Global Humanitarian Technology Conference (GHTC)*, 2021.

[24] S. Maleki, C. Fu, A. Banotra, and Z. Zong, "Understanding the impact of object oriented programming and design patterns on energy efficiency," in *2017 Eighth International Green and Sustainable Computing Conference (IGSC)*, 2017, pp. 1–6.

[25] S. Nanz and C. A. Furia, "A comparative study of programming languages in Rosetta Code," in *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, vol. 1, 2015, pp. 778–788, arXiv:1409.0252.

[26] S. Georgiou, M. Kechagia, T. Sharma, F. Sarro, and Y. Zou, "Green AI: Do deep learning frameworks have different costs?" in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 1082–1094.

[27] N. Marini, L. Pampaloni, F. Di Martino, R. Verdecchia, and E. Vicario, "Green AI: Which programming language consumes the most?" in *2025 IEEE/ACM International Workshop on Green and Sustainable Software (GREENS)*, 2025, arXiv:2501.14776.

[28] N. Alizadeh and F. Castor, "Green AI: A preliminary empirical study on energy consumption in DL models across different runtime infrastructures," in *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering — Software Engineering for AI (CAIN)*, 2024, pp. 134–139.

[29] H. Li, G. K. Rajbahadur, and C.-P. Bezemer, "Studying the impact of TensorFlow and PyTorch bindings on machine learning software quality," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 1, pp. 1–31, 2024.

[30] M. Weber, C. Kaltenecker, F. Sattler, S. Apel, and N. Siegmund, "Twins or false friends? A study on energy consumption and performance of configurable software," in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2098–2110.

[31] J. Dodge, T. Prewitt, R. Tachet des Combes, E. Odmark, R. Schwartz, E. Strubell, A. S. Luccioni, N. A. Smith, N. DeCario, and W. Buchanan, "Measuring the carbon intensity of AI in cloud instances," in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2022, pp. 1877–1894.

[32] R. Rua and J. Saraiva, "A large-scale empirical study on mobile performance: energy, run-time and memory," *Empirical Software Engineering*, vol. 29, no. 1, p. 31, 2024.

[33] N. van Kempen, H.-J. Kwon, D. T. Nguyen, and E. D. Berger, "It's not easy being green: On the energy efficiency of programming languages," in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.

[34] B. Courty, V. Schmidt, S. Luccioni, G. Kamal, M. Coutarel, B. Feld, M. Lécauchois, K. Lottick *et al.*, "mlco2/codecarbon: v3.3.0," 2026. [Online]. Available: https://github.com/mlco2/codecarbon

[35] E. Paula, J. Soni, H. Upadhyay, and L. Lagos, "Comparative analysis of model compression techniques for achieving carbon efficient AI," *Scientific Reports*, vol. 15, p. 23461, 2025.

[36] T. Almog, "An analysis of optimizer choice on energy efficiency and performance in neural network training," *arXiv preprint arXiv:2509.13516*, 2025. [Online]. Available: https://arxiv.org/abs/2509.13516

[37] A. S. Luccioni, Y. Jernite, and E. Strubell, "Power hungry processing: Watts driving the cost of AI deployment?" in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*. Rio de Janeiro, Brazil: Association for Computing Machinery, 2024, pp. 85–99.

[38] K. C. Aashish *et al.*, "Towards eco friendly cybersecurity: machine learning based anomaly detection with carbon and energy metrics," *arXiv preprint arXiv:2601.00893*, 2026, instruments Logistic Regression, Random Forest, SVM, Isolation Forest and XGBoost with CodeCarbon and defines an F1-per-kilowatt-hour eco-efficiency index.

[39] S. Aquino-Brítez, P. García-Sánchez, A. Ortiz, and D. Aquino-Brítez, "Towards an energy consumption index for deep learning models: A comparative analysis of architectures, GPUs, and measurement tools," *Sensors*, vol. 25, no. 3, p. 846, 2025, compares OpenZmeter hardware sensors against CarbonTracker and CodeCarbon on AlexNet, ResNet-18, VGG-16 and Swin Transformer.

[40] L. Bouza, A. Bugeau, and L. Lannelongue, "How to estimate carbon footprint when training deep learning models? A guide and review," *Environmental Research Communications*, vol. 5, no. 11, p. 115014, 2023.

[41] R. Fischer, "Ground-truthing AI energy consumption: validating CodeCarbon against external measurements," *it - Information Technology*, vol. 67, no. 4-6, pp. 155–163, 2026, reports estimation errors of up to 40% for established software estimators against meter ground truth.

[42] D. Li, X. Chen, M. Becchi, and Z. Zong, "Evaluating the energy efficiency of deep convolutional neural networks on CPUs and GPUs," in *Proceedings of the 2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom)*, 2016, pp. 477–484.

[43] M. Jay, V. Ostapenco, L. Lefèvre, D. Trystram, A.-C. Orgerie, and B. Fichel, "An experimental comparison of software-based power meters: Focus on CPU and GPU," in *Proceedings of the 23rd IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023, pp. 106–118.

[44] R. Chattaraj, S. Chimalakonda, V. S. Sharma, and V. Kaulgud, "Who wins the race? (R vs Python) — an exploratory study on energy consumption of machine learning algorithms," *arXiv preprint arXiv:2508.17344*, 2025.

[45] R. Feldt and A. Magazinius, "Validity threats in empirical software engineering research – an initial survey," in *Proceedings of the 22nd International Conference on Software Engineering & Knowledge Engineering (SEKE)*, 2010, pp. 374–379.

[46] A. Ampatzoglou, S. Bibi, P. Avgeriou, M. Verbeek, and A. Chatzigeorgiou, "Identifying, categorizing and mitigating threats to validity in software engineering secondary studies," *Information and Software Technology*, vol. 106, pp. 201–230, 2019.

[47] D. I. K. Sjøberg and G. R. Bergersen, "Construct validity in software engineering," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1374–1396, 2023.

[48] R. Verdecchia, E. Engström, P. Lago, P. Runeson, and Q. Song, "Threats to validity in software engineering research: A critical reflection," *Information and Software Technology*, vol. 164, p. 107329, 2023.

[49] V. R. Basili, G. Caldiera, and H. D. Rombach, "The goal question metric approach," in *Encyclopedia of Software Engineering*, vol. 1. John Wiley & Sons, 1994, pp. 528–532.

[50] T. D. Cook and D. T. Campbell, *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Boston: Houghton Mifflin, 1979.

[51] D. T. Campbell and J. C. Stanley, *Experimental and Quasi-Experimental Designs for Research*. Boston: Houghton Mifflin, 1963.

[52] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778, arXiv:1512.03385.

[53] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *3rd International Conference on Learning Representations (ICLR)*, 2015, arXiv:1409.1556.

[54] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[55] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[56] Y. Le and X. S. Yang, "Tiny ImageNet visual recognition challenge," Stanford University, Tech. Rep. CS231N course report, 2015. [Online]. Available: http://cs231n.stanford.edu/reports/2015/pdfs/yle_project.pdf

[57] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in action: Experiences in using RAPL for power measurements," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 3, no. 2, pp. 9:1–9:26, 2018.

Leonardo Pampaloni graduated with a Master's degree in Software Engineering from the University of Florence, Italy, achieving top marks. His research interests lie in software sustainability and energy efficiency in Artificial Intelligence, with particular emphasis on Green AI and computer vision. He has contributed to empirical studies on cross-language benchmarking of AI frameworks and optimization of DL workloads. More information is available at `https://github.com/Pampaj7`.

Marco Pagliocca received a Bachelor's degree in Computer Engineering with highest honors from the University of Florence, Italy, where he is currently pursuing a Master's degree in the same field. His research interests focus on environmental sustainability, human-centered engineering, and the use of technology as a means to improve everyday life. He believes that Artificial Intelligence can simplify mechanical tasks and encourage greater attention to meaningful aspects of human activity.

Enrico Vicario is a Professor of Computer Science and Engineering and the Head of the Software Technologies Laboratory of the University of Florence, Italy. His research interests include the area of software engineering, with a focus on quantitative evaluation of stochastic models, software architectures and methodologies, and on their connection through model-driven engineering practices.

Roberto Verdecchia is an Assistant Professor at the Software Technologies Laboratory (STLab) of the University of Florence, Italy. He holds a double Ph.D. in Computer Science, appointed by the Gran Sasso Science Institute, L'Aquila, Italy, and the Vrije Universiteit Amsterdam, the Netherlands. His research interests focus on the adoption of empirical methods to improve software development and system evolution, with particular interest in the fields of technical debt, software architecture, software sustainability, and software testing. More information is available at robertoverdecchia.github.io.